VISVESVARAYA TECHNOLOGICAL UNIVERSITY

"JnanaSangama", Belgaum -590014, Karnataka.

LAB REPORT

on

DATA STRUCTURES (23CS3PCDST)

Submitted by

ANANYA RANJITH (1BM24CS041)

in partial fulfillment for the award of the degree

BACHELOR OF ENGINEERING
in
COMPUTER SCIENCE AND ENGINEERING

B.M.S. COLLEGE OF ENGINEERING
(Autonomous Institution under VTU)
BENGALURU-560019
August-December 2025

B. M. S. College of Engineering,
Bull Temple Road, Bangalore 560019
(Affiliated To Visvesvaraya Technological University, Belgaum)
Department of Computer Science and Engineering

This is to certify that the Lab work entitled **"DATA STRUCTURES"** carried out by ***ANANYA RANJITH (1BM24CS041)***, who is a bonafide student of **B. M. S. College of Engineering**. It is in partial fulfillment for the award of **Bachelor of Engineering in Computer Science and Engineering** of the Visvesvaraya Technological University, Belgaum during the year 2025-2026. The Lab report has been approved as it satisfies the academic requirements in respect of Data structures Lab - **(23CS3PCDST)** work prescribed for the said degree.

Manjula S
Assistant Professor
Department of CSE
BMSCE, Bengaluru

Dr. Kavitha Sooda
Professor and Head
Department of CSE
BMSCE, Bengaluru

INDEX

Course outcomes:

CO1	Apply the concept of linear and nonlinear data structures.
CO2	Analyze data structure operations for a given problem
CO3	Design and develop solutions using the operations of linear and nonlinear data structure for a given specification.
CO4	Conduct practical experiments for demonstrating the operations of different data structures.

Lab program 1:

Write a program to simulate the working of stack using an array with the following:

a) Push

b) Pop

c) Display

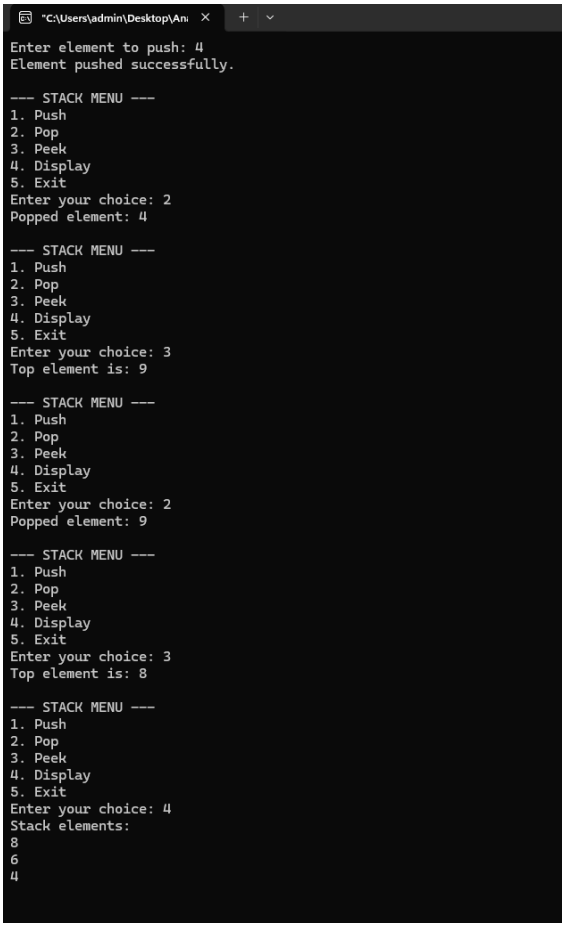
The program should print appropriate messages for stack overflow, stack underflow.

```
#include <stdio.h>
#define MAX 5
int stack[MAX];
int top = -1;

void push() {
   int item;
   if (top == MAX - 1) {
      printf("Stack Overflow! Cannot push.\n");
   } else {
      printf("Enter element to push: ");
      scanf("%d", &item);
      stack[++top] = item;
      printf("Element pushed successfully.\n");
   }
}

void pop() {
   if (top == -1) {
      printf("Stack Underflow! Stack is empty.\n");
   } else {
      printf("Popped element: %d\n", stack[top--]);
   }
}

void peek() {
   if (top == -1) {
      printf("Stack is empty. Nothing to peek.\n");
   } else {
      printf("Top element is: %d\n", stack[top]);
   }
}

void display() {
   if (top == -1) {
      printf("Stack is empty.\n");
   } else {
      printf("Stack elements:\n");
```

```
        for (int i = top; i >= 0; i--) {
            printf("%d\n", stack[i]);
        }
    }
}

int main() {
    int choice;

    do {
        printf("\n STACK MENU \n");
        printf("1. Push\n");
        printf("2. Pop\n");
        printf("3. Peek\n");
        printf("4. Display\n");
        printf("5. Exit\n");
        printf("Enter your choice: ");
        scanf("%d", &choice);

        switch (choice) {
            case 1: push(); break;
            case 2: pop(); break;
            case 3: peek(); break;
            case 4: display(); break;
            case 5: printf("Exiting program.\n"); break;
            default: printf("Invalid choice!\n");
        }
    } while (choice != 5);

    return 0;
}
```

Output:


"C:\Users\admin\Desktop\An
--- STACK MENU ---
1. Push
2. Pop
3. Peek
4. Display
5. Exit
Enter your choice: 1
Enter element to push: 4
Element pushed successfully.
--- STACK MENU ---
1. Push
2. Pop
3. Peek
4. Display
5. Exit
Enter your choice: 1
Enter element to push: 6
Element pushed successfully.
--- STACK MENU ---
1. Push
2. Pop
3. Peek
4. Display
5. Exit
Enter your choice: 1
Enter element to push: 8
Element pushed successfully.
--- STACK MENU ---
1. Push
2. Pop
3. Peek
4. Display
5. Exit
Enter your choice: 1
Enter element to push: 9
Element pushed successfully.
--- STACK MENU ---
1. Push
2. Pop
3. Peek
4. Display
5. Exit
Enter your choice: 1
Enter element to push: 4
Element pushed successfully.


"C:\Users\admin\Desktop\An
Enter element to push: 4
Element pushed successfully.
--- STACK MENU ---
1. Push
2. Pop
3. Peek
4. Display
5. Exit
Enter your choice: 2
Popped element: 4
--- STACK MENU ---
1. Push
2. Pop
3. Peek
4. Display
5. Exit
Enter your choice: 3
Top element is: 9
--- STACK MENU ---
1. Push
2. Pop
3. Peek
4. Display
5. Exit
Enter your choice: 2
Popped element: 9
--- STACK MENU ---
1. Push
2. Pop
3. Peek
4. Display
5. Exit
Enter your choice: 3
Top element is: 8
--- STACK MENU ---
1. Push
2. Pop
3. Peek
4. Display
5. Exit
Enter your choice: 4
Stack elements:
8
6
4


"C:\Users\admin\Desktop\An
3. Peek
4. Display
5. Exit
Enter your choice: 2
Popped element: 9
--- STACK MENU ---
1. Push
2. Pop
3. Peek
4. Display
5. Exit
Enter your choice: 3
Top element is: 8
--- STACK MENU ---
1. Push
2. Pop
3. Peek
4. Display
5. Exit
Enter your choice: 4
Stack elements:
8
6
4
--- STACK MENU ---
1. Push
2. Pop
3. Peek
4. Display
5. Exit
Enter your choice: 5
Exiting program.
Process returned 0 (0x0) execution time : 71.430 s
Press any key to continue.

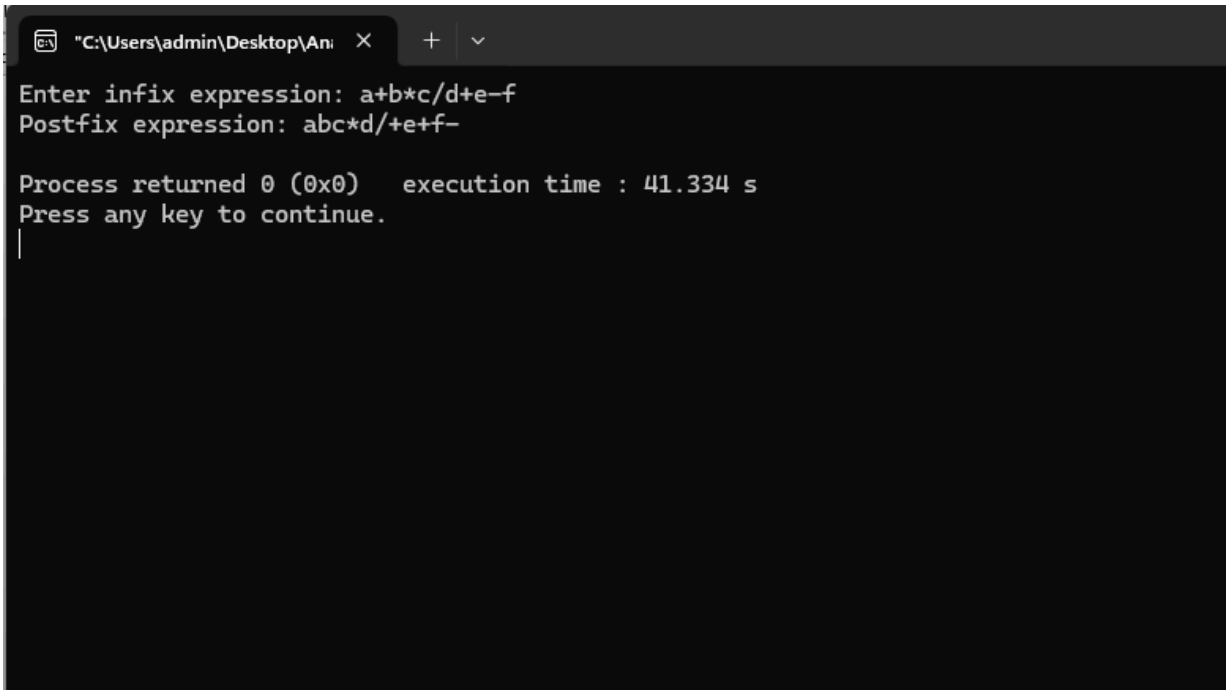
Lab program 2:

WAP to convert a given valid parenthesized infix arithmetic expression to postfix expression. The expression consists of single character operands and the binary operators + (plus), - (minus), * (multiply) and / (divide)

```
#include <stdio.h>
#include <stdlib.h>
#define MAX 100

char stack[MAX];
int top = -1;

void push(char ch) {
   stack[++top] = ch;
}

char pop() {
   return stack[top--];
}

int precedence(char ch) {
   if (ch == '+' || ch == '-')
      return 1;
   else if (ch == '*' || ch == '/')
      return 2;
   else
      return 0;
}

int main() {
   char infix[MAX], postfix[MAX];
   int i = 0, j = 0;
   char ch;

   printf("Enter infix expression: ");
   scanf("%s", infix);

   while ((ch = infix[i++]) != '\0') {

         if (isalnum(ch)) {
      postfix[j++] = ch;
      }

         else if (ch == '(') {
      push(ch);
      }
```

```
        else if (ch == ')') {
            while (stack[top] != '(') {
                postfix[j++] = pop();
            }
            pop();        }


        else {
            while (top != -1 && precedence(stack[top]) >= precedence(ch)) {
                postfix[j++] = pop();
            }
            push(ch);
        }
    }

    while (top != -1) {
        postfix[j++] = pop();
    }

    postfix[j] = '\0';

    printf("Postfix expression: %s\n", postfix);

    return 0;
}
```

Output:



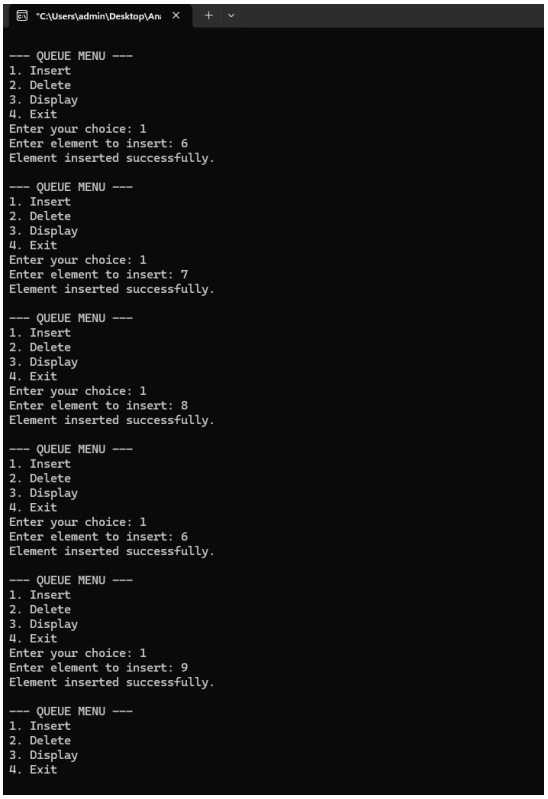
Lab program 3a:

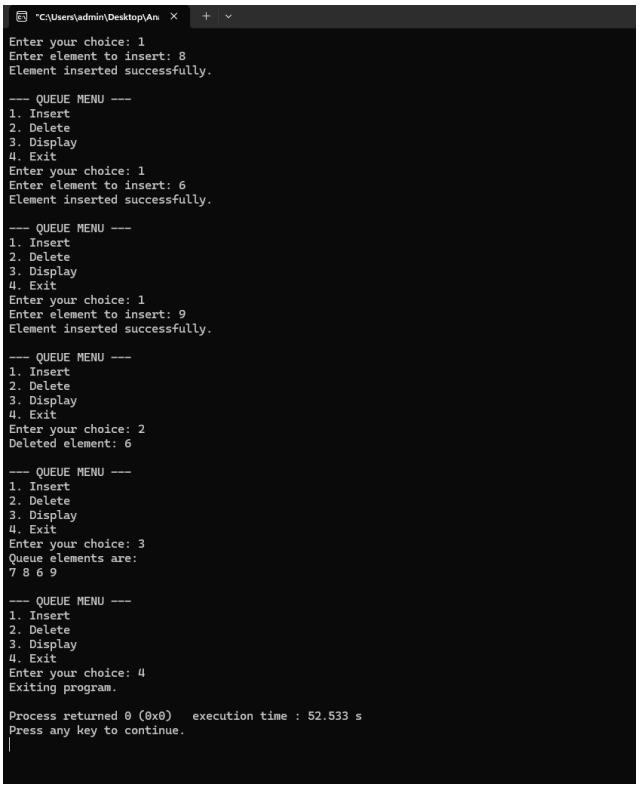
WAP to simulate the working of a queue of integers using an array. Provide the following operations: Insert, Delete, Display The program should print appropriate messages for queue empty and queue overflow conditions.

```
#include <stdio.h>
#define MAX 5

int queue[MAX];
int front = -1, rear = -1;

void insert() {
   int item;
   if (rear == MAX - 1) {
      printf("Queue Overflow! Cannot insert.\n");
   } else {
      if (front == -1)
         front = 0;
      printf("Enter element to insert: ");
      scanf("%d", &item);
      queue[++rear] = item;
      printf("Element inserted successfully.\n");
   }
}

void delete() {
   if (front == -1 || front > rear) {
      printf("Queue is Empty! Cannot delete.\n");
   } else {
      printf("Deleted element: %d\n", queue[front++]);
   }
}

void display() {
   if (front == -1 || front > rear) {
      printf("Queue is Empty.\n");
   } else {
      printf("Queue elements are:\n");
      for (int i = front; i <= rear; i++) {
         printf("%d ", queue[i]);
      }
      printf("\n");
   }
}
```

```
int main() {
    int choice;

    do {
        printf("\nQUEUE MENU \n");
        printf("1. Insert\n");
        printf("2. Delete\n");
        printf("3. Display\n");
        printf("4. Exit\n");
        printf("Enter your choice: ");
        scanf("%d", &choice);

        switch (choice) {
            case 1: insert(); break;
            case 2: delete(); break;
            case 3: display(); break;
            case 4: printf("Exiting program.\n"); break;
            default: printf("Invalid choice!\n");
        }
    } while (choice != 4);

    return 0;
}
```

Output:





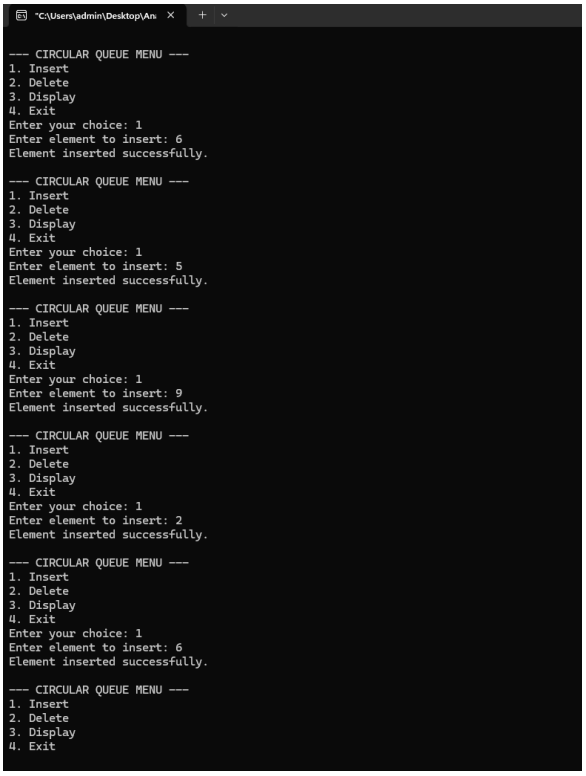
Lab program 3b:

WAP to simulate the working of a circular queue of integers using an array. Provide the following operations: Insert, Delete & Display The program should print appropriate messages for queue empty and queue overflow conditions.

```
#include <stdio.h>
#define MAX 5
int cq[MAX];
int front = -1, rear = -1;

void insert() {
   int item;
   if ((rear + 1) % MAX == front) {
      printf("Queue Overflow! Circular Queue is full.\n");
      return;
   }

   if (front == -1) {   // first insertion
      front = 0;
      rear = 0;
   } else {
      rear = (rear + 1) % MAX;
   }

   printf("Enter element to insert: ");
   scanf("%d", &item);
   cq[rear] = item;
   printf("Element inserted successfully.\n");
}

void delete() {
   if (front == -1) {
      printf("Queue Underflow! Circular Queue is empty.\n");
      return;
   }

   printf("Deleted element: %d\n", cq[front]);

   if (front == rear) {   // only one element
      front = -1;
      rear = -1;
   } else {
      front = (front + 1) % MAX;
   }
}
```

```
void display() {
    int i;

    if (front == -1) {
        printf("Queue is empty.\n");
        return;
    }

    printf("Circular Queue elements:\n");
    i = front;
    while (1) {
        printf("%d ", cq[i]);
        if (i == rear)
            break;
        i = (i + 1) % MAX;
    }
    printf("\n");
}

int main() {
    int choice;

    do {
        printf("\n CIRCULAR QUEUE MENU \n");
        printf("1. Insert\n");
        printf("2. Delete\n");
        printf("3. Display\n");
        printf("4. Exit\n");
        printf("Enter your choice: ");
        scanf("%d", &choice);

        switch (choice) {
            case 1: insert(); break;
            case 2: delete(); break;
            case 3: display(); break;
            case 4: printf("Exiting program.\n"); break;
            default: printf("Invalid choice!\n");
        }
    } while (choice != 4);

    return 0;
}
```

Output:


"C:\Users\admin\Desktop\An
--- CIRCULAR QUEUE MENU ---
1. Insert
2. Delete
3. Display
4. Exit
Enter your choice: 1
Enter element to insert: 6
Element inserted successfully.
--- CIRCULAR QUEUE MENU ---
1. Insert
2. Delete
3. Display
4. Exit
Enter your choice: 1
Enter element to insert: 5
Element inserted successfully.
--- CIRCULAR QUEUE MENU ---
1. Insert
2. Delete
3. Display
4. Exit
Enter your choice: 1
Enter element to insert: 9
Element inserted successfully.
--- CIRCULAR QUEUE MENU ---
1. Insert
2. Delete
3. Display
4. Exit
Enter your choice: 1
Enter element to insert: 2
Element inserted successfully.
--- CIRCULAR QUEUE MENU ---
1. Insert
2. Delete
3. Display
4. Exit
Enter your choice: 1
Enter element to insert: 6
Element inserted successfully.
--- CIRCULAR QUEUE MENU ---
1. Insert
2. Delete
3. Display
4. Exit


"C:\Users\admin\Desktop\An
--- CIRCULAR QUEUE MENU ---
1. Insert
2. Delete
3. Display
4. Exit
Enter your choice: 1
Enter element to insert: 6
Element inserted successfully.
--- CIRCULAR QUEUE MENU ---
1. Insert
2. Delete
3. Display
4. Exit
Enter your choice: 1
Queue Overflow! Circular Queue is full.
--- CIRCULAR QUEUE MENU ---
1. Insert
2. Delete
3. Display
4. Exit
Enter your choice: 1
Queue Overflow! Circular Queue is full.
--- CIRCULAR QUEUE MENU ---
1. Insert
2. Delete
3. Display
4. Exit
Enter your choice: 7
Invalid choice!
--- CIRCULAR QUEUE MENU ---
1. Insert
2. Delete
3. Display
4. Exit
Enter your choice: 2
Deleted element: 6
--- CIRCULAR QUEUE MENU ---
1. Insert
2. Delete
3. Display
4. Exit
Enter your choice: 3
Circular Queue elements:
5 9 2 6


"C:\Users\admin\Desktop\An
3. Display
4. Exit
Enter your choice: 1
Queue Overflow! Circular Queue is full.
--- CIRCULAR QUEUE MENU ---
1. Insert
2. Delete
3. Display
4. Exit
Enter your choice: 7
Invalid choice!
--- CIRCULAR QUEUE MENU ---
1. Insert
2. Delete
3. Display
4. Exit
Enter your choice: 2
Deleted element: 6
--- CIRCULAR QUEUE MENU ---
1. Insert
2. Delete
3. Display
4. Exit
Enter your choice: 3
Circular Queue elements:
5 9 2 6
--- CIRCULAR QUEUE MENU ---
1. Insert
2. Delete
3. Display
4. Exit
Enter your choice: 4
Exiting program.
Process returned 0 (0x0)   execution time : 46.683 s
Press any key to continue.

Lab program 4a:

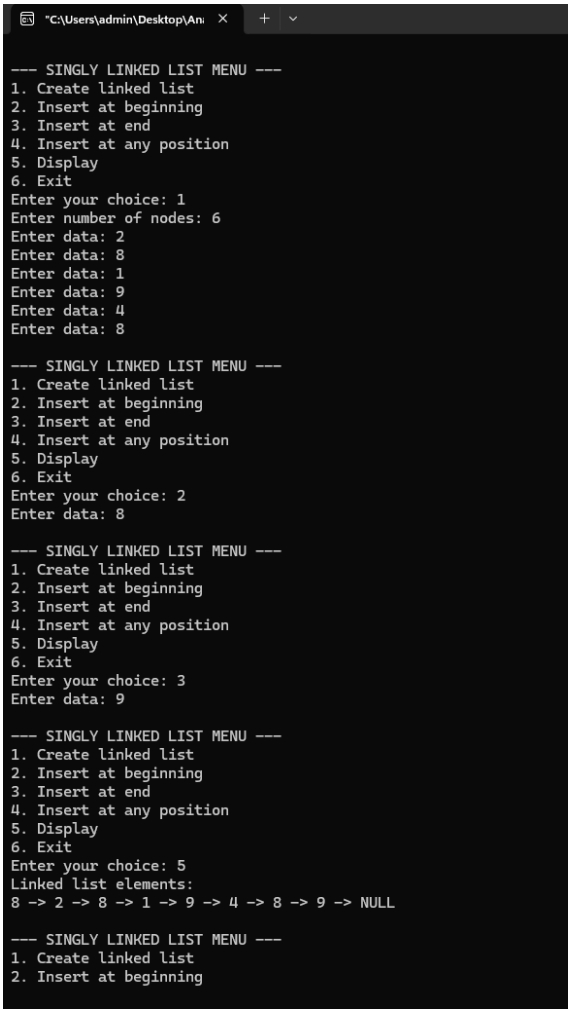
WAP to Implement Singly Linked List with following operations a) Createalinkedlist. b) Insertion of a node at first position, at any position and at end of list. Display the contents of the linked list.

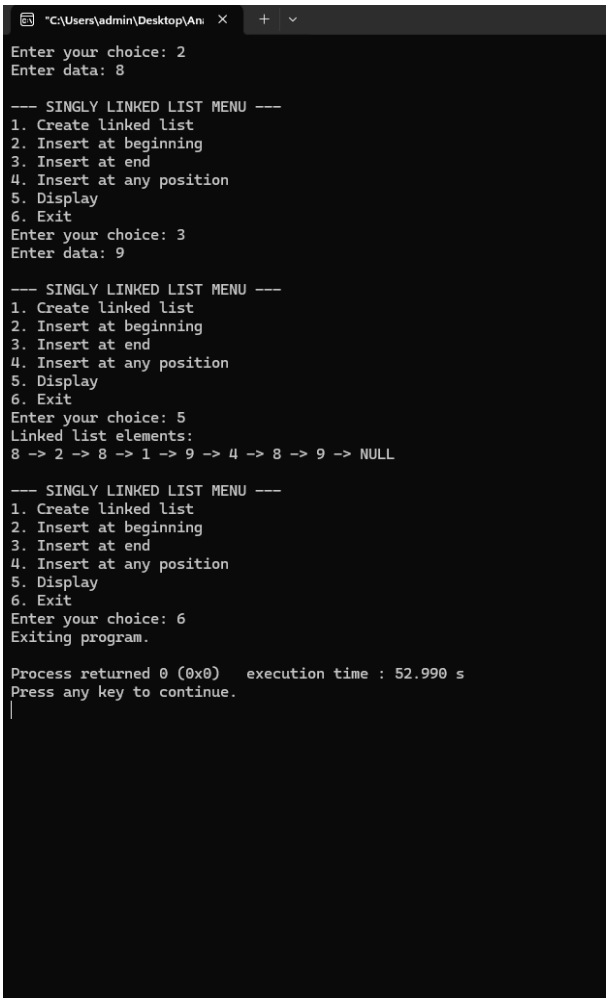
```
#include <stdio.h>
#include <stdlib.h>

struct node {
   int data;
   struct node *next;
};
struct node *head = NULL;

void create() {
   struct node *newnode, *temp;
   int n, i;

   printf("Enter number of nodes: ");
   scanf("%d", &n);

   for (i = 0; i < n; i++) {
      newnode = (struct node *)malloc(sizeof(struct node));
      printf("Enter data: ");
      scanf("%d", &newnode->data);
      newnode->next = NULL;

      if (head == NULL) {
         head = newnode;
         temp = head;
      } else {
         temp->next = newnode;
         temp = newnode;
      }
   }
}

void insert_begin() {
   struct node *newnode;
   newnode = (struct node *)malloc(sizeof(struct node));

   printf("Enter data: ");
   scanf("%d", &newnode->data);

   newnode->next = head;
   head = newnode;
```

```
}

void insert_end() {
    struct node *newnode, *temp;
    newnode = (struct node *)malloc(sizeof(struct node));

    printf("Enter data: ");
    scanf("%d", &newnode->data);
    newnode->next = NULL;

    if (head == NULL) {
        head = newnode;
    } else {
        temp = head;
        while (temp->next != NULL)
            temp = temp->next;
        temp->next = newnode;
    }
}

void insert_pos() {
    struct node *newnode, *temp;
    int pos, i;

    printf("Enter position: ");
    scanf("%d", &pos);

    if (pos == 1) {
        insert_begin();
        return;
    }

    newnode = (struct node *)malloc(sizeof(struct node));
    printf("Enter data: ");
    scanf("%d", &newnode->data);

    temp = head;
    for (i = 1; i < pos - 1 && temp != NULL; i++)
        temp = temp->next;

    if (temp == NULL) {
        printf("Invalid position!\n");
        free(newnode);
    } else {
        newnode->next = temp->next;
        temp->next = newnode;
    }
```

```
}

void display() {
   struct node *temp;
   if (head == NULL) {
      printf("Linked list is empty.\n");
      return;
   }

   temp = head;
   printf("Linked list elements:\n");
   while (temp != NULL) {
      printf("%d -> ", temp->data);
      temp = temp->next;
   }
   printf("NULL\n");
}

int main() {
   int choice;

   do {
      printf("\n SINGLY LINKED LIST MENU \n");
      printf("1. Create linked list\n");
      printf("2. Insert at beginning\n");
      printf("3. Insert at end\n");
      printf("4. Insert at any position\n");
      printf("5. Display\n");
      printf("6. Exit\n");
      printf("Enter your choice: ");
      scanf("%d", &choice);

      switch (choice) {
         case 1: create(); break;
         case 2: insert_begin(); break;
         case 3: insert_end(); break;
         case 4: insert_pos(); break;
         case 5: display(); break;
         case 6: printf("Exiting program.\n"); break;
         default: printf("Invalid choice!\n");
      }
   } while (choice != 6);

   return 0;
}
```

Output:


"C:\Users\admin\Desktop\An
--- SINGLY LINKED LIST MENU ---
1. Create linked list
2. Insert at beginning
3. Insert at end
4. Insert at any position
5. Display
6. Exit
Enter your choice: 1
Enter number of nodes: 6
Enter data: 2
Enter data: 8
Enter data: 1
Enter data: 9
Enter data: 4
Enter data: 8
--- SINGLY LINKED LIST MENU ---
1. Create linked list
2. Insert at beginning
3. Insert at end
4. Insert at any position
5. Display
6. Exit
Enter your choice: 2
Enter data: 8
--- SINGLY LINKED LIST MENU ---
1. Create linked list
2. Insert at beginning
3. Insert at end
4. Insert at any position
5. Display
6. Exit
Enter your choice: 3
Enter data: 9
--- SINGLY LINKED LIST MENU ---
1. Create linked list
2. Insert at beginning
3. Insert at end
4. Insert at any position
5. Display
6. Exit
Enter your choice: 5
Linked list elements:
8 -> 2 -> 8 -> 1 -> 9 -> 4 -> 8 -> 9 -> NULL
--- SINGLY LINKED LIST MENU ---
1. Create linked list
2. Insert at beginning


"C:\Users\admin\Desktop\An
Enter your choice: 2
Enter data: 8
--- SINGLY LINKED LIST MENU ---
1. Create linked list
2. Insert at beginning
3. Insert at end
4. Insert at any position
5. Display
6. Exit
Enter your choice: 3
Enter data: 9
--- SINGLY LINKED LIST MENU ---
1. Create linked list
2. Insert at beginning
3. Insert at end
4. Insert at any position
5. Display
6. Exit
Enter your choice: 5
Linked list elements:
8 -> 2 -> 8 -> 1 -> 9 -> 4 -> 8 -> 9 -> NULL
--- SINGLY LINKED LIST MENU ---
1. Create linked list
2. Insert at beginning
3. Insert at end
4. Insert at any position
5. Display
6. Exit
Enter your choice: 6
Exiting program.
Process returned 0 (0x0)   execution time : 52.990 s
Press any key to continue.

Lab program 4b:

Program - Leetcode platform

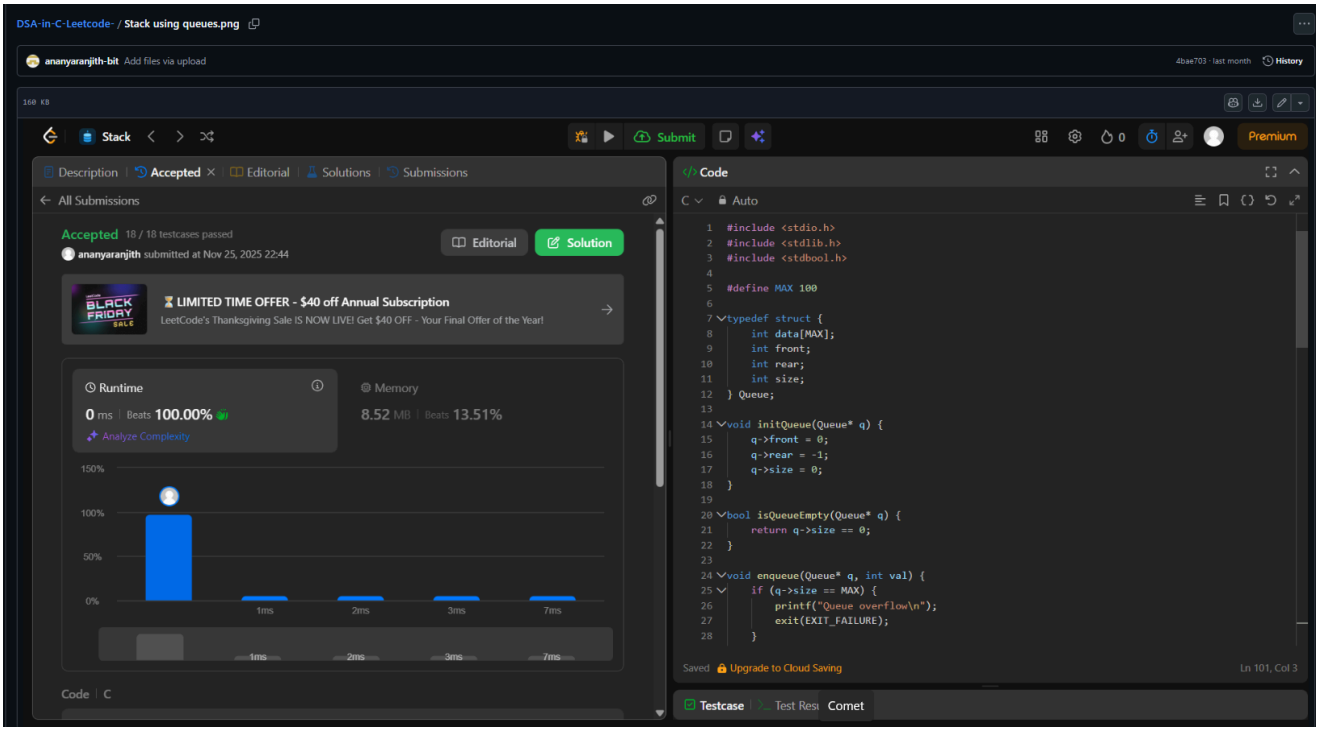
Implement Stack using Queue.

```
#include <stdio.h>
#include <stdlib.h>
#include <stdbool.h>
#define MAX 100

typedef struct {
    int data[MAX];
    int front;
    int rear;
    int size;
} Queue;

void initQueue(Queue* q) {
    q->front = 0;
    q->rear = -1;
    q->size = 0;
}

bool isQueueEmpty(Queue* q) {
    return q->size == 0;
}

void enqueue(Queue* q, int val) {
    if (q->size == MAX) {
        printf("Queue overflow\n");
        exit(EXIT_FAILURE);
    }
    q->rear = (q->rear + 1) % MAX;
    q->data[q->rear] = val;
    q->size++;
}

int dequeue(Queue* q) {
    if (isQueueEmpty(q)) {
        printf("Queue underflow\n");
        exit(EXIT_FAILURE);
```

```
    }
    int val = q->data[q->front];
    q->front = (q->front + 1) % MAX;
    q->size--;
    return val;
}

int peek(Queue* q) {
    if (isQueueEmpty(q)) {
        printf("Queue is empty\n");
        exit(EXIT_FAILURE);
    }
    return q->data[q->front];
}

typedef struct {
    Queue q;
} MyStack;


MyStack* myStackCreate() {
    MyStack* st = (MyStack*)malloc(sizeof(MyStack));
    initQueue(&st->q);
    return st;
}

void myStackPush(MyStack* obj, int x) {
    enqueue(&obj->q, x);
    int rotation = obj->q.size-1;
    while(rotation){
        enqueue(&obj->q, dequeue(&obj->q));
        rotation--;
    }
}

int myStackPop(MyStack* obj) {
    return dequeue(&obj->q);
}

int myStackTop(MyStack* obj) {
    return peek(&obj->q);
```

```
}

bool myStackEmpty(MyStack* obj) {
    return isQueueEmpty(&obj->q);
}

void myStackFree(MyStack* obj) {
    free(obj);
}

/**
 * Your MyStack struct will be instantiated and called as such:
 * MyStack* obj = myStackCreate();
 * myStackPush(obj, x);

 * int param_2 = myStackPop(obj);

 * int param_3 = myStackTop(obj);

 * bool param_4 = myStackEmpty(obj);

 * myStackFree(obj);
*/
```

Output:



Lab program 5a:

WAP to Implement Singly Linked List with following operations
a) Create a linked list.
b) Deletion of first element, specified element and last element in the list.
c) Display the contents of the linked list.

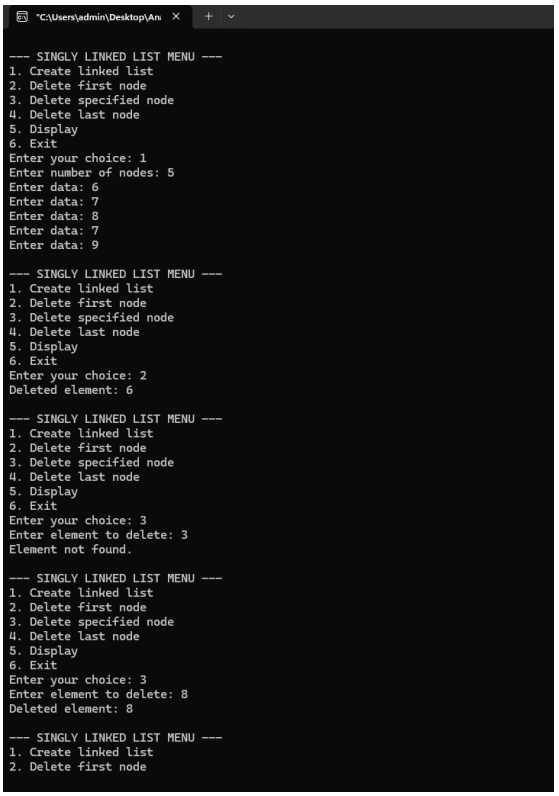
```
#include <stdio.h>
#include <stdlib.h>
// Node definition
struct node {
    int data;
    struct node *next;
};

struct node *head = NULL;

void create() {
    struct node *newnode, *temp;
    int n, i;

    printf("Enter number of nodes: ");
    scanf("%d", &n);

    for (i = 0; i < n; i++) {
        newnode = (struct node *)malloc(sizeof(struct node));
        printf("Enter data: ");
        scanf("%d", &newnode->data);
        newnode->next = NULL;

        if (head == NULL) {
            head = newnode;
            temp = head;
        } else {
            temp->next = newnode;
            temp = newnode;
        }
    }
}

void delete_first() {
    struct node *temp;
    if (head == NULL) {
        printf("List is empty. Cannot delete.\n");
        return;
    }
```

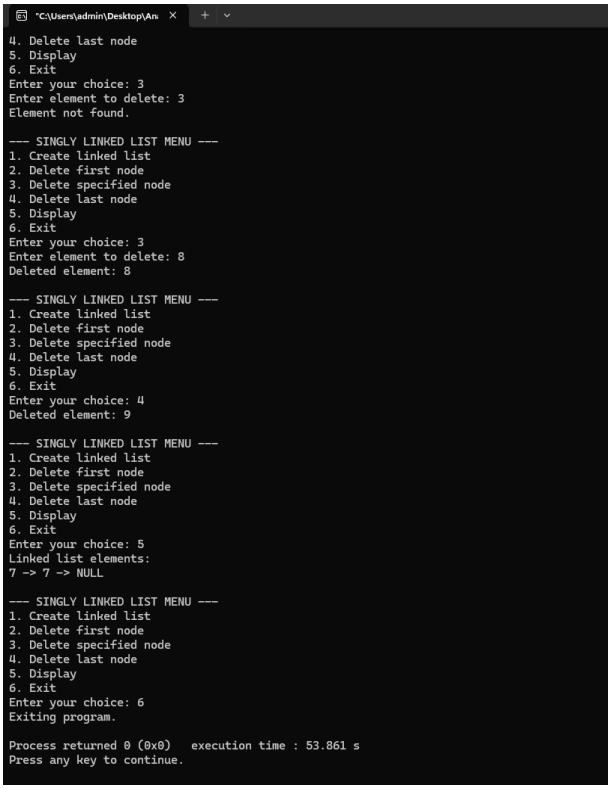
```
    temp = head;
    head = head->next;
    printf("Deleted element: %d\n", temp->data);
    free(temp);
}

void delete_last() {
    struct node *temp, *prev;
    if (head == NULL) {
        printf("List is empty. Cannot delete.\n");
        return;
    }
    if (head->next == NULL) {
        printf("Deleted element: %d\n", head->data);
        free(head);
        head = NULL;
        return;
    }

    temp = head;
    while (temp->next != NULL) {
        prev = temp;
        temp = temp->next;
    }
    prev->next = NULL;
    printf("Deleted element: %d\n", temp->data);
    free(temp);
}

void delete_specified() {
    struct node *temp, *prev;
    int key;

    if (head == NULL) {
        printf("List is empty. Cannot delete.\n");
        return;
    }

    printf("Enter element to delete: ");
    scanf("%d", &key);

    // If first node is the one to delete
    if (head->data == key) {
        delete_first();
        return;
    }
```

```
    temp = head;
    while (temp != NULL && temp->data != key) {
        prev = temp;
        temp = temp->next;
    }

    if (temp == NULL) {
        printf("Element not found.\n");
    } else {
        prev->next = temp->next;
        printf("Deleted element: %d\n", temp->data);
        free(temp);
    }
}

void display() {
    struct node *temp;
    if (head == NULL) {
        printf("Linked list is empty.\n");
        return;
    }

    printf("Linked list elements:\n");
    temp = head;
    while (temp != NULL) {
        printf("%d -> ", temp->data);
        temp = temp->next;
    }
    printf("NULL\n");
}

int main() {
    int choice;

    do {
        printf("\n--- SINGLY LINKED LIST MENU ---\n");
        printf("1. Create linked list\n");
        printf("2. Delete first node\n");
        printf("3. Delete specified node\n");
        printf("4. Delete last node\n");
        printf("5. Display\n");
        printf("6. Exit\n");
        printf("Enter your choice: ");
        scanf("%d", &choice);

        switch (choice) {
            case 1: create(); break;
```

```
            case 2: delete_first(); break;
            case 3: delete_specified(); break;
            case 4: delete_last(); break;
            case 5: display(); break;
            case 6: printf("Exiting program.\n"); break;
            default: printf("Invalid choice!\n");
        }
    } while (choice != 6);

    return 0;
}
```

Output:





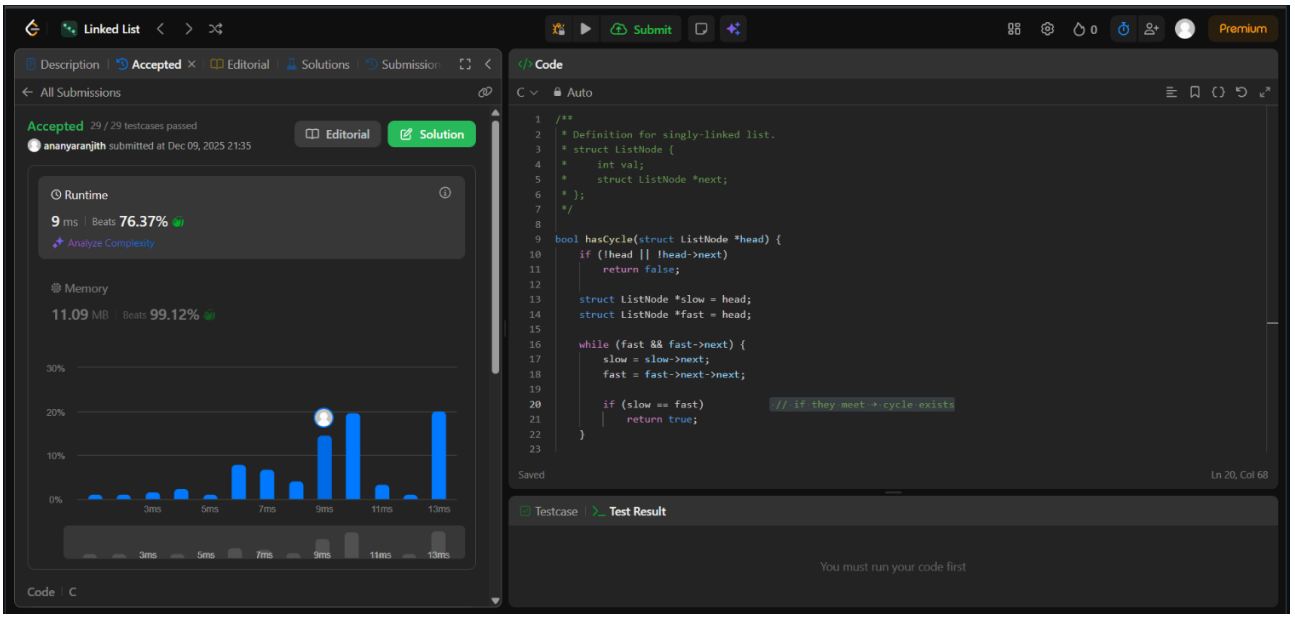
Lab program 5b:

Program - Leetcode platform

i) Linked List Cycle

```
/**
 * Definition for singly-linked list.
 * struct ListNode {
 *     int val;
 *     struct ListNode *next;
 * };
 */

bool hasCycle(struct ListNode *head) {
    if (!head || !head->next)
        return false;

    struct ListNode *slow = head;
    struct ListNode *fast = head;

    while (fast && fast->next) {
        slow = slow->next;
        fast = fast->next->next;

        if (slow == fast)           // if they meet → cycle exists
            return true;
    }

    return false; // fast reached NULL → no cycle
}
```

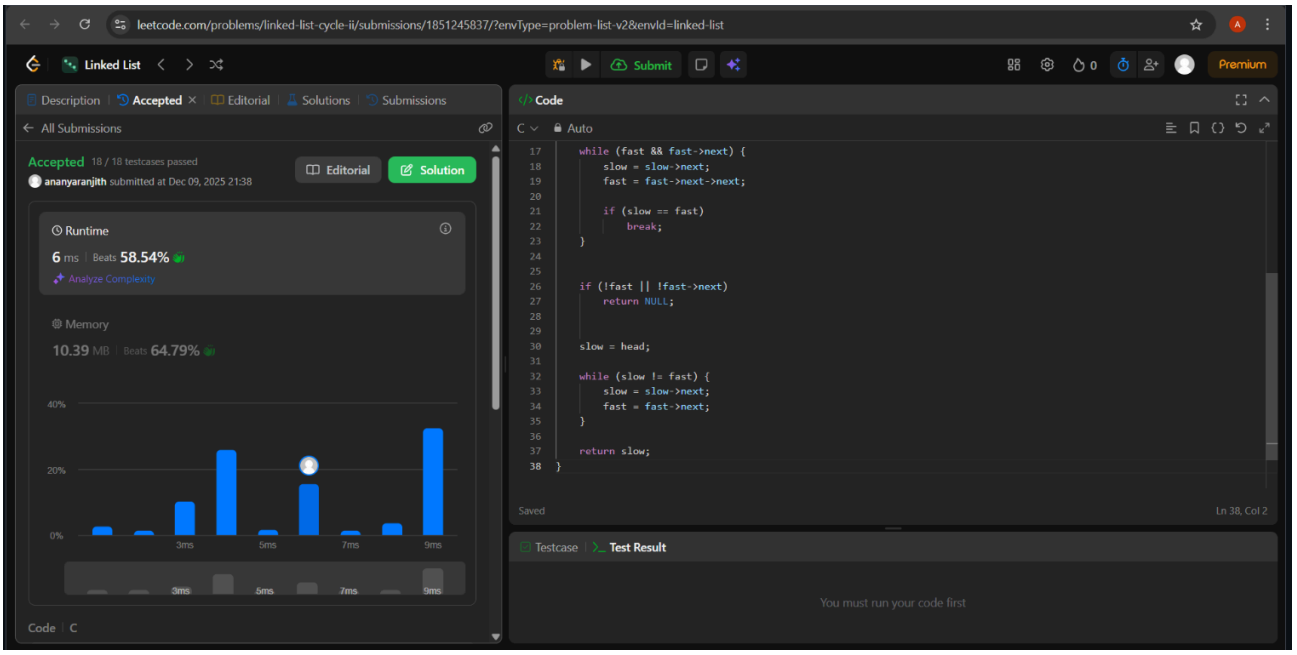
Output:


Linked List
Submit
Premium
Description
Accepted
Editorial
Solutions
Submission
All Submissions
Accepted 29 / 29 testcases passed
ananyaranjith submitted at Dec 09, 2025 21:35
Editorial
Solution
Runtime
9 ms | Beats 76.37%
Analyze Complexity
Memory
11.09 MB | Beats 99.12%
30%
20%
10%
0%
3ms
5ms
7ms
9ms
11ms
13ms
Code | C
Code
Auto
/**
* Definition for singly-linked list.
* struct ListNode {
*   int val;
*   struct ListNode *next;
* };
*/
bool hasCycle(struct ListNode *head) {
if (!head || !head->next)
return false;
struct ListNode *slow = head;
struct ListNode *fast = head;
while (fast && fast->next) {
slow = slow->next;
fast = fast->next->next;
if (slow == fast) // if they meet → cycle exists
return true;
}
Saved
Ln 20, Col 68
Testcase | Test Result
You must run your code first

ii) Linked List Cycle - 2

```
/**
 * Definition for singly-linked list.
 * struct ListNode {
 *     int val;
 *     struct ListNode *next;
 * };
 */

struct ListNode *detectCycle(struct ListNode *head) {
    if (!head || !head->next)
        return NULL;

    struct ListNode *slow = head;
    struct ListNode *fast = head;


    while (fast && fast->next) {
        slow = slow->next;
        fast = fast->next->next;

        if (slow == fast)
            break;
    }


    if (!fast || !fast->next)
        return NULL;


    slow = head;

    while (slow != fast) {
        slow = slow->next;
        fast = fast->next;
    }

    return slow;
}
```

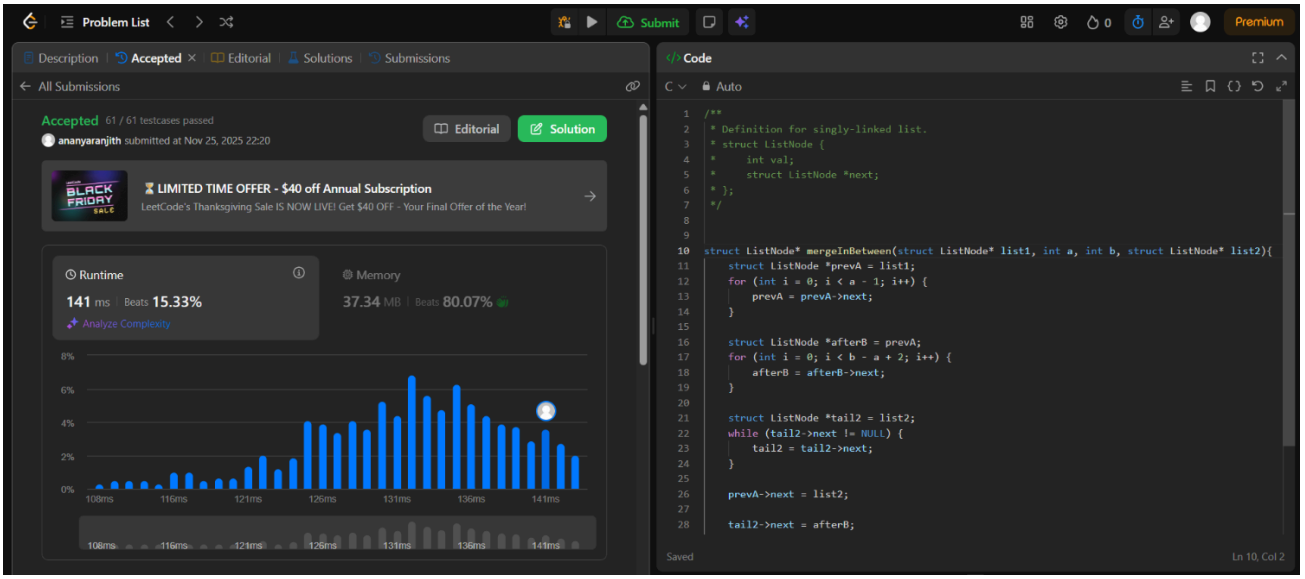
Output:


leetcode.com/problems/linked-list-cycle-ii/submissions/1851245837/?envType=problem-list-v2&envId=linked-list
Linked List
Submit
Premium
Description
Accepted
Editorial
Solutions
Submissions
All Submissions
Accepted 18 / 18 testcases passed
ananyaranjith submitted at Dec 09, 2025 21:38
Editorial
Solution
Runtime
6 ms | Beats 58.54%
Analyze Complexity
Memory
10.39 MB | Beats 64.79%
40%
20%
0%
3ms
5ms
7ms
9ms
Code | C
Code
C
Auto
while (fast && fast->next) {
slow = slow->next;
fast = fast->next->next;
if (slow == fast)
break;
}
if (!fast || !fast->next)
return NULL;
slow = head;
while (slow != fast) {
slow = slow->next;
fast = fast->next;
}
return slow;
}
Saved
Ln 38, Col 2
Testcase
Test Result
You must run your code first

iii) Implement Cycle Detection in Linked List

```
/**
 * Definition for singly-linked list.
 * struct ListNode {
 *     int val;
 *     struct ListNode *next;
 * };
 */

#include <stdio.h>
#include <stdlib.h>

bool hasCycle(struct ListNode *head) {
    if (head == NULL || head->next == NULL)
        return false;

    struct ListNode *slow = head;
    struct ListNode *fast = head;

    while (fast != NULL && fast->next != NULL) {
        slow = slow->next;          // moves 1 step
        fast = fast->next->next;    // moves 2 steps

        if (slow == fast)           // cycle detected
            return true;
    }

    return false;   // fast reached NULL → no cycle
}
```

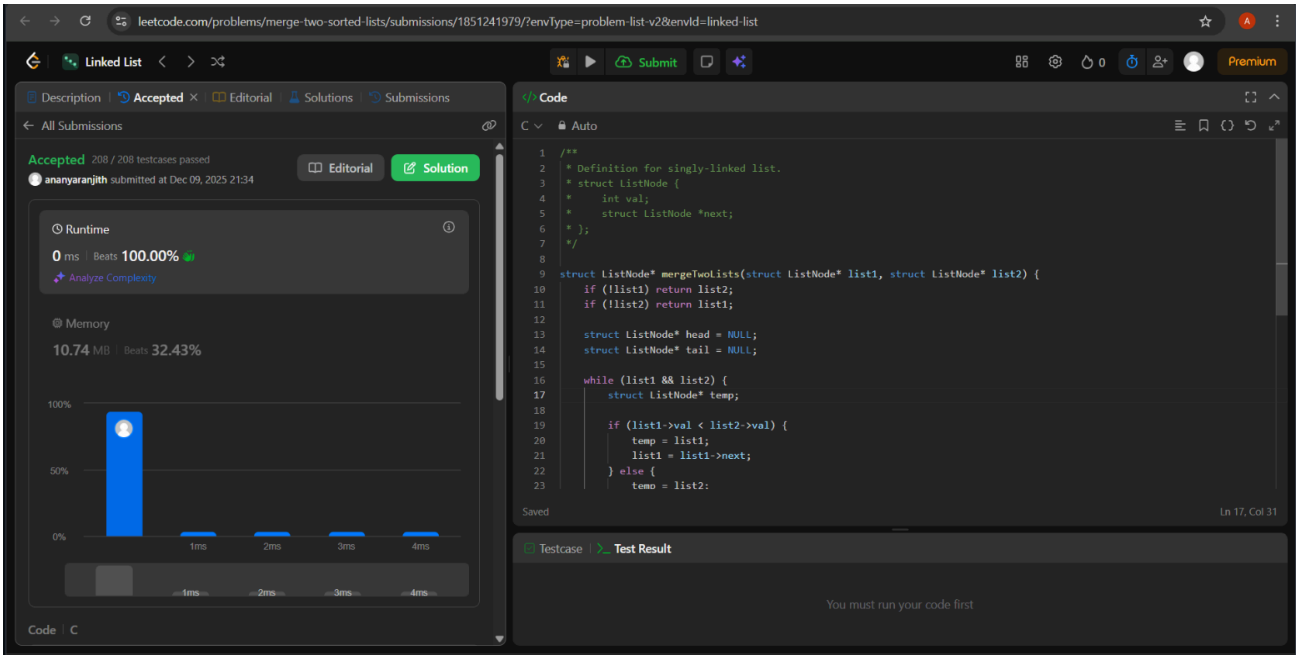
Output:


Problem List
Submit
Premium
Description
Accepted
Editorial
Solutions
Submissions
All Submissions
Accepted 61 / 61 testcases passed
ananyaranjith submitted at Nov 25, 2025 22:20
Editorial
Solution
LIMITED TIME OFFER - $40 off Annual Subscription
LeetCode's Thanksgiving Sale IS NOW LIVE! Get $40 OFF - Your Final Offer of the Year!
Runtime
141 ms | Beats 15.33%
Analyze Complexity
Memory
37.34 MB | Beats 80.07%
8%
6%
4%
2%
0%
108ms
116ms
121ms
126ms
131ms
136ms
141ms
Code
C
Auto
/**
* Definition for singly-linked list.
* struct ListNode {
*   int val;
*   struct ListNode *next;
* };
*/
struct ListNode* mergeInBetween(struct ListNode* list1, int a, int b, struct ListNode* list2){
struct ListNode *prevA = list1;
for (int i = 0; i < a - 1; i++) {
prevA = prevA->next;
}
struct ListNode *afterB = prevA;
for (int i = 0; i < b - a + 2; i++) {
afterB = afterB->next;
}
struct ListNode *tail2 = list2;
while (tail2->next != NULL) {
tail2 = tail2->next;
}
prevA->next = list2;
tail2->next = afterB;
Saved
Ln 10, Col 2

iv) Merge two Sorted Linked Lists

```
/**
 * Definition for singly-linked list.
 * struct ListNode {
 *     int val;
 *     struct ListNode *next;
 * };
 */

struct ListNode* mergeTwoLists(struct ListNode* list1, struct ListNode* list2) {
    if (!list1) return list2;
    if (!list2) return list1;

    struct ListNode* head = NULL;
    struct ListNode* tail = NULL;

    while (list1 && list2) {
        struct ListNode* temp;

        if (list1->val < list2->val) {
            temp = list1;
            list1 = list1->next;
        } else {
            temp = list2;
            list2 = list2->next;
        }

        if (!head) {
            head = tail = temp;
        } else {
            tail->next = temp;
            tail = temp;
        }
    }

    if (list1) tail->next = list1;
    else tail->next = list2;

    return head;
}
```

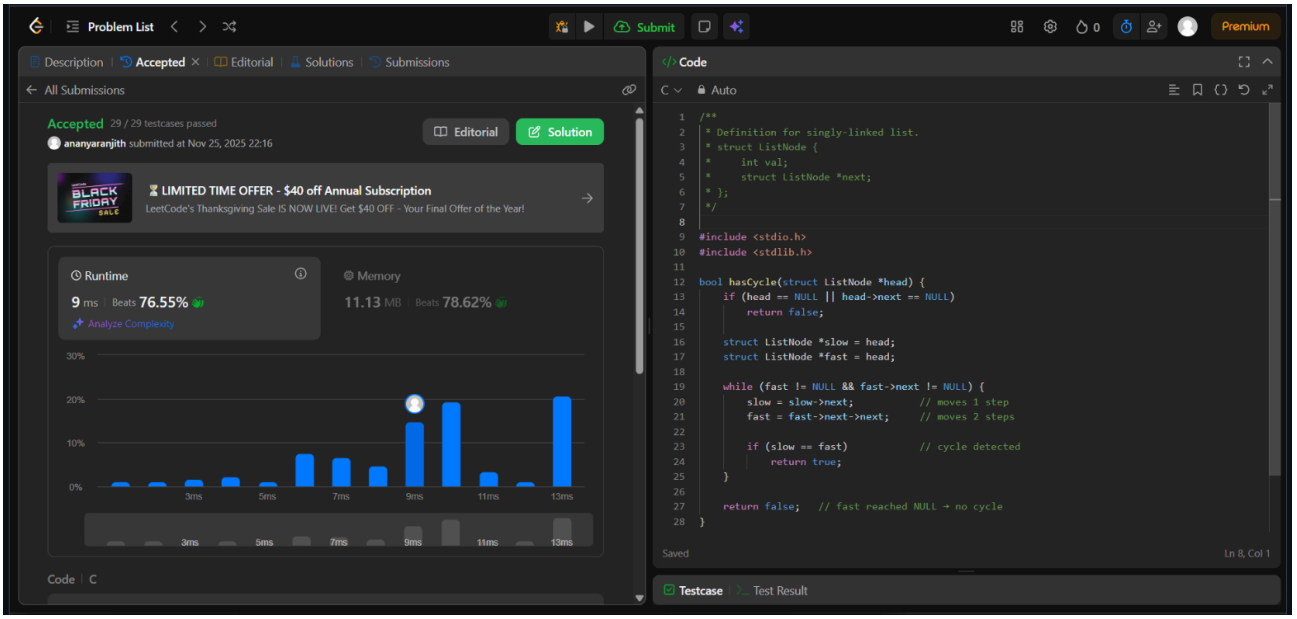
Output:


leetcode.com/problems/merge-two-sorted-lists/submissions/1851241979/?envType=problem-list-v2&envId=linked-list
Linked List
Submit
Premium
Description
Accepted
Editorial
Solutions
Submissions
All Submissions
Accepted 208 / 208 testcases passed
ananyaranjith submitted at Dec 09, 2025 21:34
Editorial
Solution
Runtime
0 ms | Beats 100.00%
Analyze Complexity
Memory
10.74 MB | Beats 32.43%
100%
50%
0%
1ms
2ms
3ms
4ms
Code | C
Code
C
Auto
/**
* Definition for singly-linked list.
* struct ListNode {
* int val;
* struct ListNode *next;
* };
*/
struct ListNode* mergeTwoLists(struct ListNode* list1, struct ListNode* list2) {
if (!list1) return list2;
if (!list2) return list1;
struct ListNode* head = NULL;
struct ListNode* tail = NULL;
while (list1 && list2) {
struct ListNode* temp;
if (list1->val < list2->val) {
temp = list1;
list1 = list1->next;
} else {
temo = list2:
Saved
Ln 17, Col 31
Testcase
Test Result
You must run your code first

v) Merge in Between Linked Lists

```
/**
 * Definition for singly-linked list.
 * struct ListNode {
 *     int val;
 *     struct ListNode *next;
 * };
 */


struct ListNode* mergeInBetween(struct ListNode* list1, int a, int b, struct ListNode* list2){
    struct ListNode *prevA = list1;
    for (int i = 0; i < a - 1; i++) {
        prevA = prevA->next;
    }

    struct ListNode *afterB = prevA;
    for (int i = 0; i < b - a + 2; i++) {
        afterB = afterB->next;
    }

    struct ListNode *tail2 = list2;
    while (tail2->next != NULL) {
        tail2 = tail2->next;
    }

    prevA->next = list2;

    tail2->next = afterB;

    return list1;
}
```

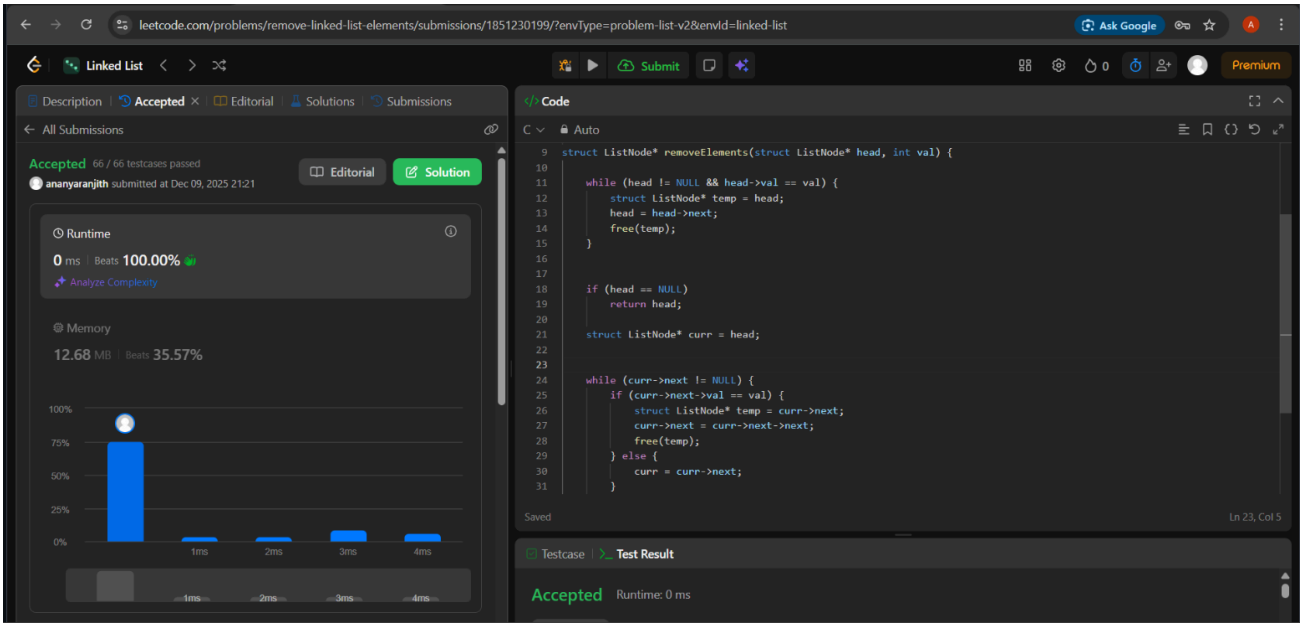
Output:


Problem List
Submit
Premium
Description
Accepted
Editorial
Solutions
Submissions
All Submissions
Accepted 29 / 29 testcases passed
ananyaranjith submitted at Nov 25, 2025 22:16
Editorial
Solution
LIMITED TIME OFFER - $40 off Annual Subscription
LeetCode's Thanksgiving Sale IS NOW LIVE! Get $40 OFF - Your Final Offer of the Year!
Runtime
9 ms | Beats 76.55%
Analyze Complexity
Memory
11.13 MB | Beats 78.62%
30%
20%
10%
0%
3ms
5ms
7ms
9ms
11ms
13ms
Code | C
Code
C
Auto
/**
* Definition for singly-linked list.
* struct ListNode {
* int val;
* struct ListNode *next;
* };
*/
#include <stdio.h>
#include <stdlib.h>
bool hasCycle(struct ListNode *head) {
if (head == NULL || head->next == NULL)
return false;
struct ListNode *slow = head;
struct ListNode *fast = head;
while (fast != NULL && fast->next != NULL) {
slow = slow->next; // moves 1 step
fast = fast->next->next; // moves 2 steps
if (slow == fast) // cycle detected
return true;
}
return false; // fast reached NULL → no cycle
}
Saved
Ln 8, Col 1
Testcase
Test Result

vi) Remove Linked List Elements

```
/**
 * Definition for singly-linked list.
 * struct ListNode {
 *     int val;
 *     struct ListNode *next;
 * };
 */

struct ListNode* removeElements(struct ListNode* head, int val) {

    while (head != NULL && head->val == val) {
        struct ListNode* temp = head;
        head = head->next;
        free(temp);
    }


    if (head == NULL)
        return head;

    struct ListNode* curr = head;


    while (curr->next != NULL) {
        if (curr->next->val == val) {
            struct ListNode* temp = curr->next;
            curr->next = curr->next->next;
            free(temp);
        } else {
            curr = curr->next;
        }
    }

    return head;
}
```

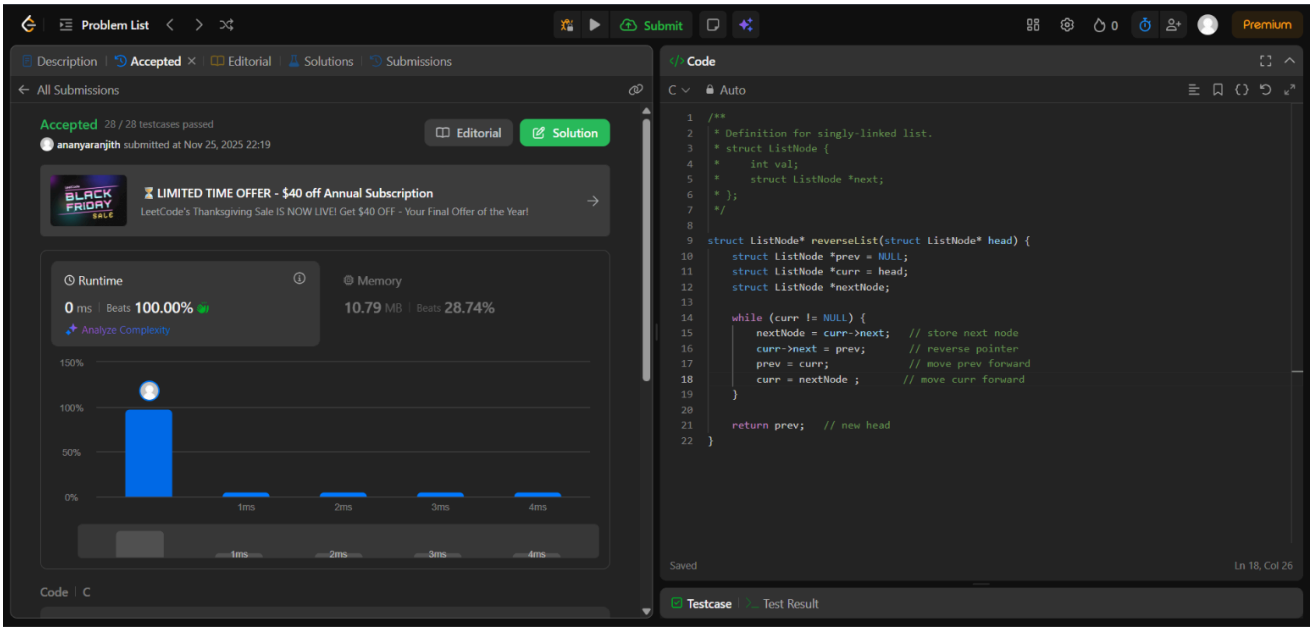
Output:


leetcode.com/problems/remove-linked-list-elements/submissions/1851230199/?envType=problem-list-v2&envId=linked-list
Ask Google
Linked List
Submit
Premium
Description
Accepted
Editorial
Solutions
Submissions
All Submissions
Accepted 66 / 66 testcases passed
ananyaranjith submitted at Dec 09, 2025 21:21
Editorial
Solution
Runtime
0 ms | Beats 100.00%
Analyze Complexity
Memory
12.68 MB | Beats 35.57%
100%
75%
50%
25%
0%
1ms
2ms
3ms
4ms
Code
C
Auto
struct ListNode* removeElements(struct ListNode* head, int val) {
while (head != NULL && head->val == val) {
struct ListNode* temp = head;
head = head->next;
free(temp);
}
if (head == NULL)
return head;
struct ListNode* curr = head;
while (curr->next != NULL) {
if (curr->next->val == val) {
struct ListNode* temp = curr->next;
curr->next = curr->next->next;
free(temp);
} else {
curr = curr->next;
}
Saved
Ln 23, Col 5
Testcase
Test Result
Accepted
Runtime: 0 ms

vii) Reverse Linked List

```
/**
 * Definition for singly-linked list.
 * struct ListNode {
 *     int val;
 *     struct ListNode *next;
 * };
 */

struct ListNode* reverseList(struct ListNode* head) {
    struct ListNode *prev = NULL;
    struct ListNode *curr = head;
    struct ListNode *nextNode;

    while (curr != NULL) {
        nextNode = curr->next;   // store next node
        curr->next = prev;       // reverse pointer
        prev = curr;             // move prev forward
        curr = nextNode ;        // move curr forward
    }

    return prev;   // new head
}
```

Output:

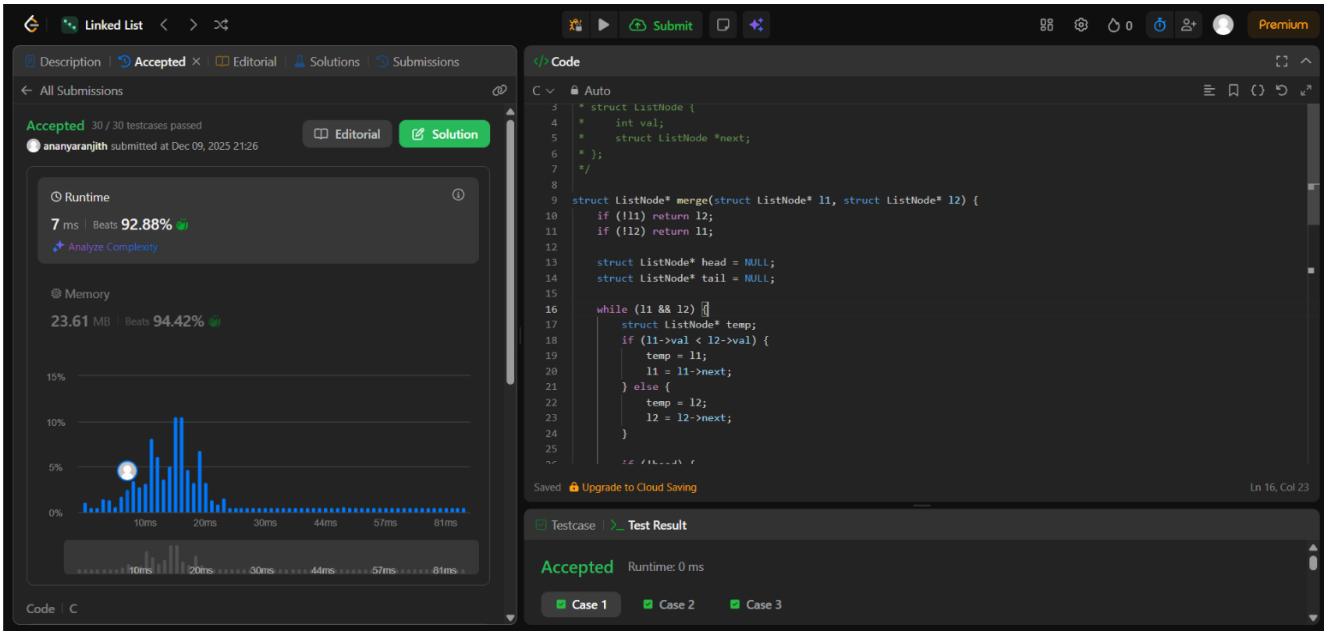


viii) Sort Linked List

```
/**
 * Definition for singly-linked list.
 * struct ListNode {
 *     int val;
 *     struct ListNode *next;
 * };
 */

struct ListNode* merge(struct ListNode* l1, struct ListNode* l2) {
    if (!l1) return l2;
    if (!l2) return l1;

    struct ListNode* head = NULL;
    struct ListNode* tail = NULL;

    while (l1 && l2) {
        struct ListNode* temp;
        if (l1->val < l2->val) {
            temp = l1;
            l1 = l1->next;
        } else {
            temp = l2;
            l2 = l2->next;
        }

        if (!head) {
            head = tail = temp;
        } else {
            tail->next = temp;
            tail = temp;
        }
    }

    if (l1) tail->next = l1;
    if (l2) tail->next = l2;

    return head;
}
```

```
struct ListNode* sortList(struct ListNode* head) {

    if (!head || !head->next)
        return head;


    struct ListNode *slow = head, *fast = head, *prev = NULL;

    while (fast && fast->next) {
        prev = slow;
        slow = slow->next;
        fast = fast->next->next;
    }


    prev->next = NULL;


    struct ListNode* left = sortList(head);
    struct ListNode* right = sortList(slow);


    return merge(left, right);
}
```

Output:



Lab program 6a:

WAP to Implement Single Link List with following operations:
Sort the linked list, Reverse the linked list, Concatenation of two linked lists.

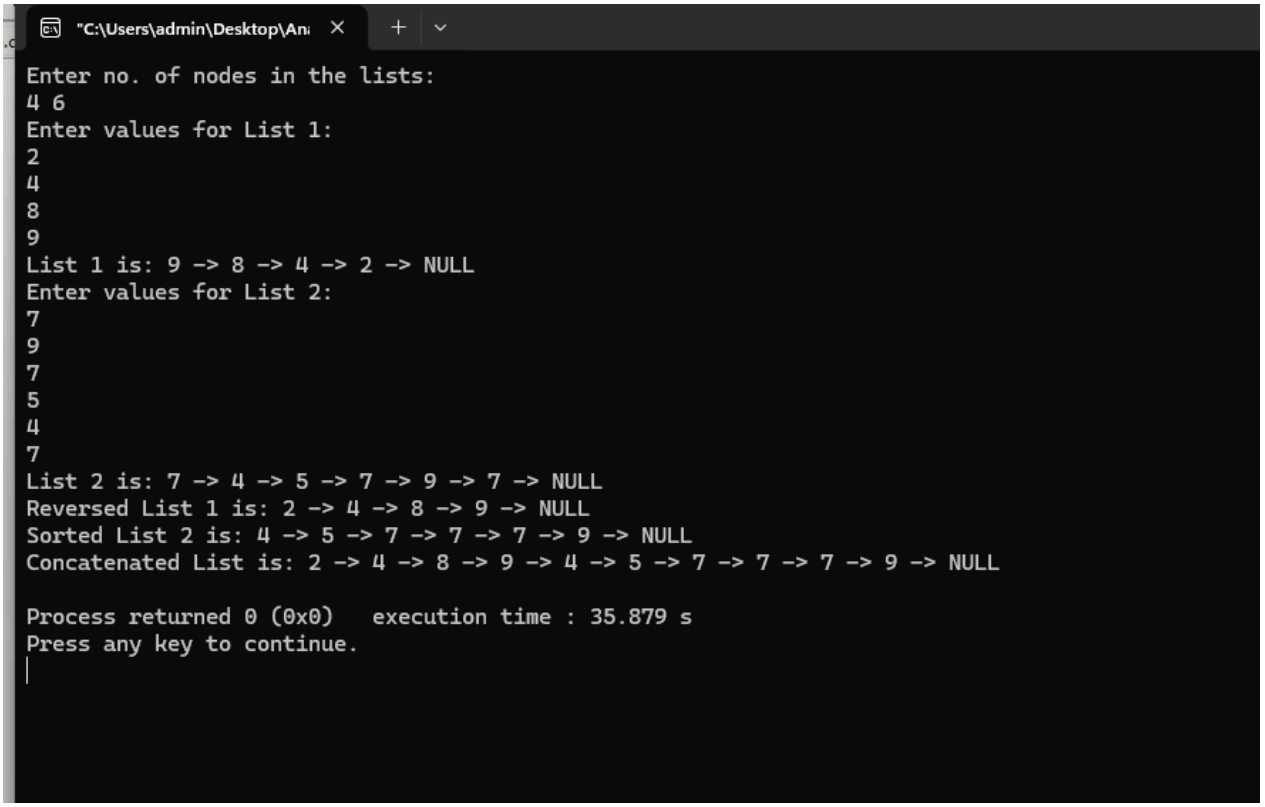
```
#include <stdio.h>
#include <stdlib.h>

struct node {
   int data;
   struct node *next;
};

struct node *create(int value) {
   struct node *temp;
   temp = (struct node *)malloc(sizeof(struct node));

   if (temp == NULL) {
      printf("Memory allocation failed\n");
      exit(0);
   }

   temp->data = value;
   temp->next = NULL;
   return temp;
}

struct node *insert_beg(struct node *head, int value) {
   struct node *newnode;
   newnode = create(value);

   newnode->next = head;
   head = newnode;

   return head;
}

void display(struct node *head) {
   struct node *current = head;
   while (current != NULL) {
      printf("%d -> ", current->data);
      current = current->next;
   }
   printf("NULL\n");
}
```

```
struct node *reverse_list(struct node *head) {
    struct node *prev = NULL;
    struct node *current = head;
    struct node *next = NULL;

    while (current != NULL) {
        next = current->next;
        current->next = prev;
        prev = current;
        current = next;
    }
    return prev;
}

struct node *sort_list(struct node *head) {
    int swapped;
    struct node *ptr1;
    struct node *lptr = NULL;

    if (head == NULL)
        return head;

    do {
        swapped = 0;
        ptr1 = head;

        while (ptr1->next != lptr) {
            if (ptr1->data > ptr1->next->data) {
                int temp = ptr1->data;
                ptr1->data = ptr1->next->data;
                ptr1->next->data = temp;
                swapped = 1;
            }
            ptr1 = ptr1->next;
        }
        lptr = ptr1;
    } while (swapped);

    return head;
}

struct node *concat_lists(struct node *head1, struct node *head2) {
    struct node *current;

    if (head1 == NULL)
        return head2;
```

```
    if (head2 == NULL)
        return head1;

    current = head1;
    while (current->next != NULL) {
        current = current->next;
    }

    current->next = head2;
    return head1;
}

int main() {
    struct node *head1 = NULL;
    struct node *head2 = NULL;
    int m, n, val, i;

    printf("Enter no. of nodes in the lists:\n");
    scanf("%d %d", &m, &n);

    printf("Enter values for List 1:\n");
    for (i = 0; i < m; i++) {
        scanf("%d", &val);
        head1 = insert_beg(head1, val);
    }

    printf("List 1 is: ");
    display(head1);

    printf("Enter values for List 2:\n");
    for (i = 0; i < n; i++) {
        scanf("%d", &val);
        head2 = insert_beg(head2, val);
    }

    printf("List 2 is: ");
    display(head2);

    head1 = reverse_list(head1);
    printf("Reversed List 1 is: ");
    display(head1);

    head2 = sort_list(head2);
    printf("Sorted List 2 is: ");
    display(head2);
```

```
    struct node *concat_head = concat_lists(head1, head2);
    printf("Concatenated List is: ");
    display(concat_head);

    return 0;
}
```

Output:



Lab program 6b:

WAP to Implement Single Link List to simulate Stack & Queue Operations.

```
#include <stdio.h>
#include <stdlib.h>
struct node
{
int data;
struct node *next;
};

struct node *stack_top = NULL;
struct node *queue_front = NULL;
struct node *queue_rear = NULL;

void push(int value)
{
struct node *newnode = (struct node*)
malloc(sizeof(struct node));
if (newnode == NULL)
{
printf("Memory allocation failed!\n");
return;
}
newnode->data = value;
newnode->next = stack_top;
stack_top = newnode;
printf("%d pushed to stack.\n", value);
}

int pop()
{
if (stack_top == NULL)
{
printf("Stack Underflow: Stack is empty.\n");
return;
}

struct node *temp = stack_top;
int val = temp->data;
stack_top = stack_top->next;
```

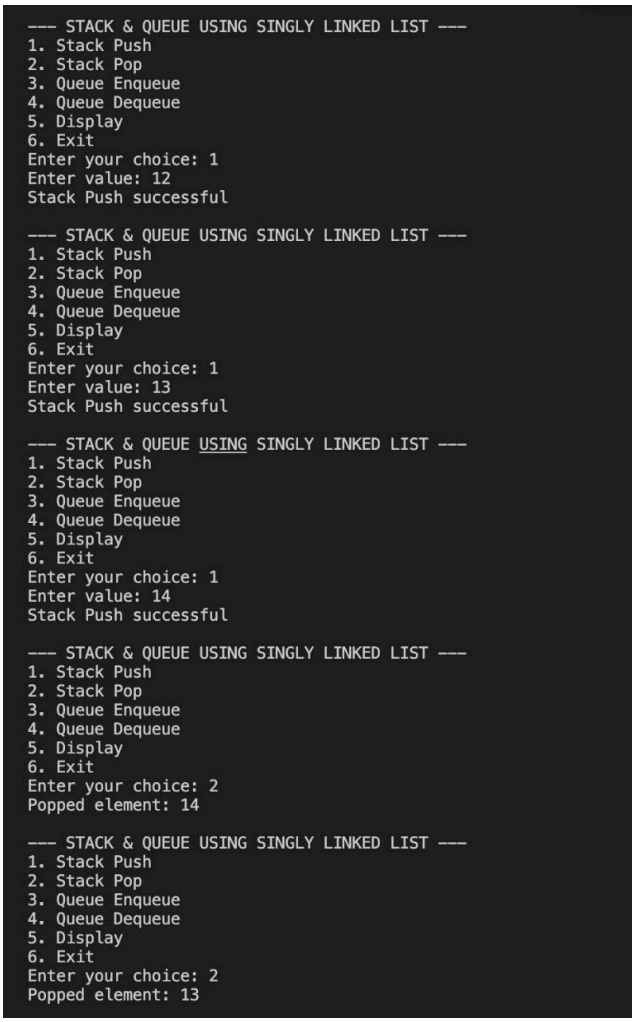
```
free(temp);
printf("%d popped from stack.\n", val);
return val;
}

int peek_stack()
{
if (stack_top == NULL)
{
printf("Stack is empty. No top element.\n");
return -1;
}
return stack_top->data;
}

void display_stack()
{
if (stack_top == NULL)
{
printf("Stack is empty.\n");
return;
}
struct node *current = stack_top;
printf("Stack elements: ");
while (current != NULL)
{
printf("%d ", current->data);
current = current->next;
}
printf("\n");
}

void enqueue(int value)
{
struct node *newnode = (struct node*)malloc(sizeof(struct node));
if (newnode == NULL)
{
printf("Memory allocation failed!\n");
return;
}
newnode->data = value;
```

```
newnode->next = NULL;

if (queue_front == NULL)
{
queue_front = newnode;
queue_rear = newnode;
}
else
{
queue_rear->next = newnode;
queue_rear = newnode;
}
printf("%d enqueued to queue.\n", value);
}

int dequeue()
{
if (queue_front == NULL)
{
printf("Queue Underflow: Queue is empty.\n");
return;
}
struct node *temp = queue_front;
int value = temp->data;
queue_front = queue_front->next;
if (queue_front == NULL)
{
queue_rear = NULL; // If queue becomes empty after dequeue
}

free(temp);
printf("%d dequeued from queue.\n", value);
return value;
}

int peek_queue()
{
if (queue_front == NULL)
{
printf("Queue is empty. No front element.\n");
return -1;
```

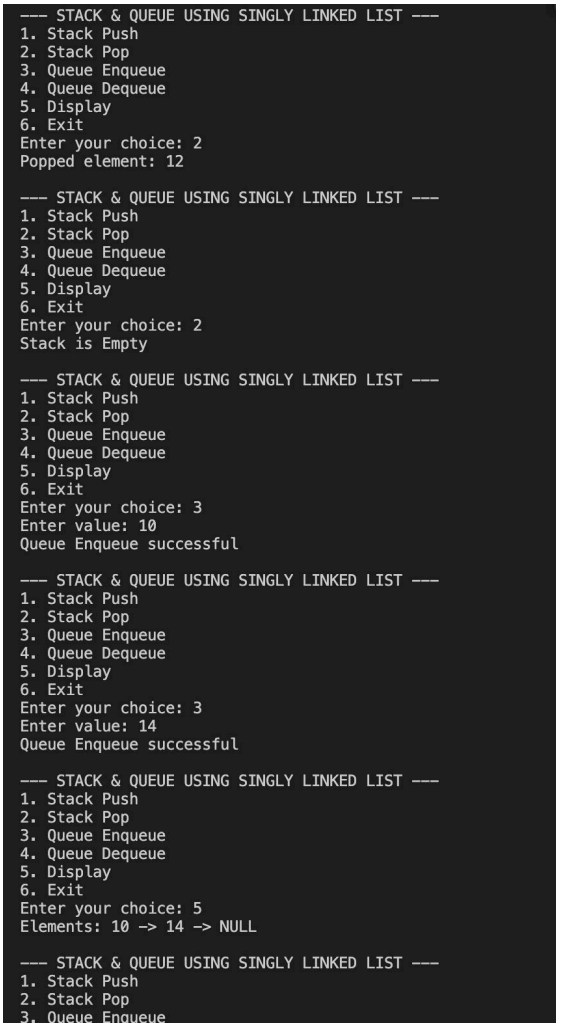
```
}
return queue_front->data;
}

void display_queue()
{
if (queue_front == NULL)
{
printf("Queue is empty.\n");
return;
}
struct node* current = queue_front;
printf("Queue elements: ");
while (current != NULL)
{
printf("%d ", current->data);
current = current->next;
}
printf("\n");
}

int main()
int option, val, del, p;
do
{
printf("\n MAIN MENU ");
printf("\n 1. Push an element");
printf("\n 2. Pop an element");
printf("\n 3. Peek an element");
printf("\n 4. Display the Stack");
printf("\n 5. Enqueue an element");
printf("\n 6. Dequeue an element");
printf("\n 7. Peek an element");
printf("\n 8. Display the queue");
printf("\n 9. EXIT");
printf("\n Enter your option : ");
scanf("%d", &option);
switch(option)
{
case 1:
printf("\n Enter the number to be inserted into stack: ");
```

```
scanf("%d", &val);
push(val);
break;
case 2:
del = pop();
printf("\n The number popped is : %d", del);
break;
case 3:
p = peek_stack();
printf("\n The number peeked is : %d", p);
break;
case 4:
printf("\n The elements in the stack \n”);
display_stack();
break;
case 5: printf("\n Enter the number to be inserted in the queue : “);
scanf("%d", &val);
enqueue(val);
break;
Case 6:
del = dequeue():
printf("\n The number dequeued is : %d", del);
break;
Case 7:
p= peek_queue()
printf("\n The number peeked is : %d", p);
break;
case 8:
printf("\n The elements in the queue \n”);
display_queue();
break;
}
} while(option!=9);
return 0;
}
```

Output:



```
--- STACK & QUEUE USING SINGLY LINKED LIST ---
1. Stack Push
2. Stack Pop
3. Queue Enqueue
4. Queue Dequeue
5. Display
6. Exit
Enter your choice: 1
Enter value: 12
Stack Push successful

--- STACK & QUEUE USING SINGLY LINKED LIST ---
1. Stack Push
2. Stack Pop
3. Queue Enqueue
4. Queue Dequeue
5. Display
6. Exit
Enter your choice: 1
Enter value: 13
Stack Push successful

--- STACK & QUEUE USING SINGLY LINKED LIST ---
1. Stack Push
2. Stack Pop
3. Queue Enqueue
4. Queue Dequeue
5. Display
6. Exit
Enter your choice: 1
Enter value: 14
Stack Push successful

--- STACK & QUEUE USING SINGLY LINKED LIST ---
1. Stack Push
2. Stack Pop
3. Queue Enqueue
4. Queue Dequeue
5. Display
6. Exit
Enter your choice: 2
Popped element: 14

--- STACK & QUEUE USING SINGLY LINKED LIST ---
1. Stack Push
2. Stack Pop
3. Queue Enqueue
4. Queue Dequeue
5. Display
6. Exit
Enter your choice: 2
Popped element: 13
```



```
--- STACK & QUEUE USING SINGLY LINKED LIST ---
1. Stack Push
2. Stack Pop
3. Queue Enqueue
4. Queue Dequeue
5. Display
6. Exit
Enter your choice: 2
Popped element: 12

--- STACK & QUEUE USING SINGLY LINKED LIST ---
1. Stack Push
2. Stack Pop
3. Queue Enqueue
4. Queue Dequeue
5. Display
6. Exit
Enter your choice: 2
Stack is Empty

--- STACK & QUEUE USING SINGLY LINKED LIST ---
1. Stack Push
2. Stack Pop
3. Queue Enqueue
4. Queue Dequeue
5. Display
6. Exit
Enter your choice: 3
Enter value: 10
Queue Enqueue successful

--- STACK & QUEUE USING SINGLY LINKED LIST ---
1. Stack Push
2. Stack Pop
3. Queue Enqueue
4. Queue Dequeue
5. Display
6. Exit
Enter your choice: 3
Enter value: 14
Queue Enqueue successful

--- STACK & QUEUE USING SINGLY LINKED LIST ---
1. Stack Push
2. Stack Pop
3. Queue Enqueue
4. Queue Dequeue
5. Display
6. Exit
Enter your choice: 5
Elements: 10 -> 14 -> NULL

--- STACK & QUEUE USING SINGLY LINKED LIST ---
1. Stack Push
2. Stack Pop
3. Queue Enqueue
```

Lab program 7a:

WAP to Implement doubly link list with primitive operations
a) Create a doubly linked list.
b) Insert a new node to the left of the node.
c) Delete the node based on a specific value.
d) Display the contents of the list.

```
#include <stdio.h>
#include <stdlib.h>


struct node {
    int data;
    struct node *prev;
    struct node *next;
};


struct node *head = NULL;


struct node *create(int data) {
    struct node *newnode = (struct node*)malloc(sizeof(struct node));
    if (newnode == NULL) {
        printf("Memory allocation failed!\n");
        exit(1);
    }
    newnode->data = data;
    newnode->prev = NULL;
    newnode->next = NULL;
    return newnode;
}

struct node *insert_end(int data) {
    struct node *newnode = create(data);

    if (head == NULL) {
        head = newnode;
        return head;
    }

    struct node *current = head;
    while (current->next != NULL) {
        current = current->next;
    }
```

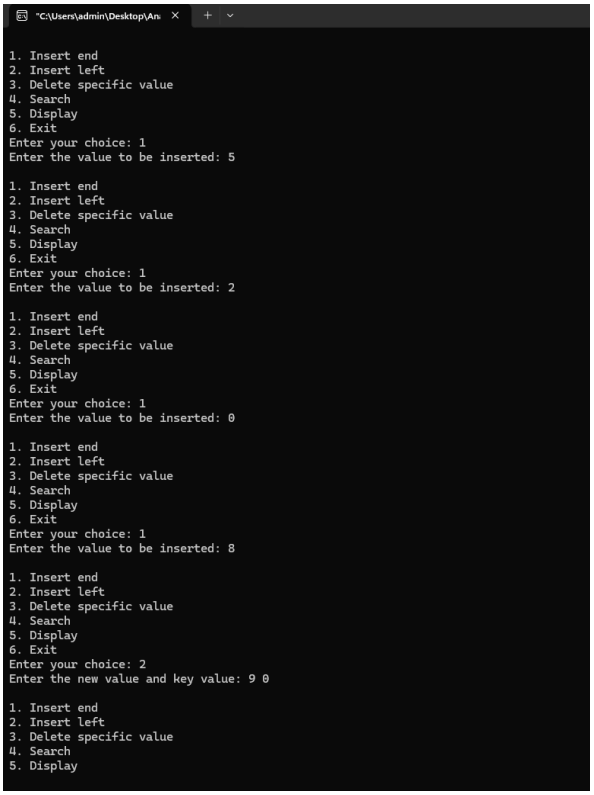
```
    current->next = newnode;
    newnode->prev = current;
    return head;
}


void insert_left(int new_data, int key_value) {
    struct node *newnode = create(new_data);

    if (head == NULL) {
        head = newnode;
        return;
    }

    struct node *current = head;
    while (current != NULL && current->data != key_value) {
        current = current->next;
    }

    if (current == NULL) {
        printf("Target node with value %d not found. Node not inserted.\n", key_value);
        free(newnode);
        return;
    }

    newnode->next = current;
    newnode->prev = current->prev;

    if (current->prev != NULL) {
        current->prev->next = newnode;
    } else {
        head = newnode;
    }

    current->prev = newnode;
}


void delete_spec(int key) {
    if (head == NULL) {
        printf("List is empty.\n");
        return;
    }

    struct node *current = head;
```

```
    while (current != NULL && current->data != key) {
        current = current->next;
    }

    if (current == NULL) {
        printf("Node with value %d not found.\n", key);
        return;
    }

    if (current->prev != NULL)
        current->prev->next = current->next;
    else
        head = current->next;

    if (current->next != NULL)
        current->next->prev = current->prev;

    free(current);
    printf("Node deleted successfully.\n");
}


void search(int key) {
    struct node *current = head;
    int pos = 0;

    while (current != NULL && current->data != key) {
        pos++;
        current = current->next;
    }

    if (current == NULL) {
        printf("Element not found\n");
    } else {
        printf("Element found at position %d\n", pos + 1);
    }
}

void display() {
    if (head == NULL) {
        printf("List is empty.\n");
        return;
    }

    struct node *current = head;
    printf("Doubly Linked List is: ");
```

```
    while (current != NULL) {
        printf("%d <-> ", current->data);
        current = current->next;
    }

    printf("NULL\n");
}

void main() {
    int ch, value, key;

    while(1) {
        printf("\n1. Insert end\n2. Insert left\n3. Delete specific value\n4. Search\n
                    5. Display\n6. Exit\n");
        printf("Enter your choice: ");
        scanf("%d", &ch);

        switch (ch) {
            case 1:
                printf("Enter the value to be inserted: ");
                scanf("%d", &value);
                insert_end(value);
                break;

            case 2:
                printf("Enter the new value and key value: ");
                scanf("%d %d", &value, &key);
                insert_left(value, key);
                break;

            case 3:
                printf("Enter the value of the node to be deleted: ");
                scanf("%d", &key);
                delete_spec(key);
                break;

            case 4:
                printf("Enter the value to search: ");
                scanf("%d", &key);
                search(key);
                break;

            case 5:
                display();
                break;

            case 6:
```

```
            printf("Exiting...\n");
            break;

        default:
            printf("Invalid choice!\n");
    }

  }
}
```

Output:



Lab program 8a:

Write a program

a) ToconstructabinarySearchtree.

b) To traverse the tree using all the methods i.e., in-order, preorder and post order

c) To display the elements in the tree.

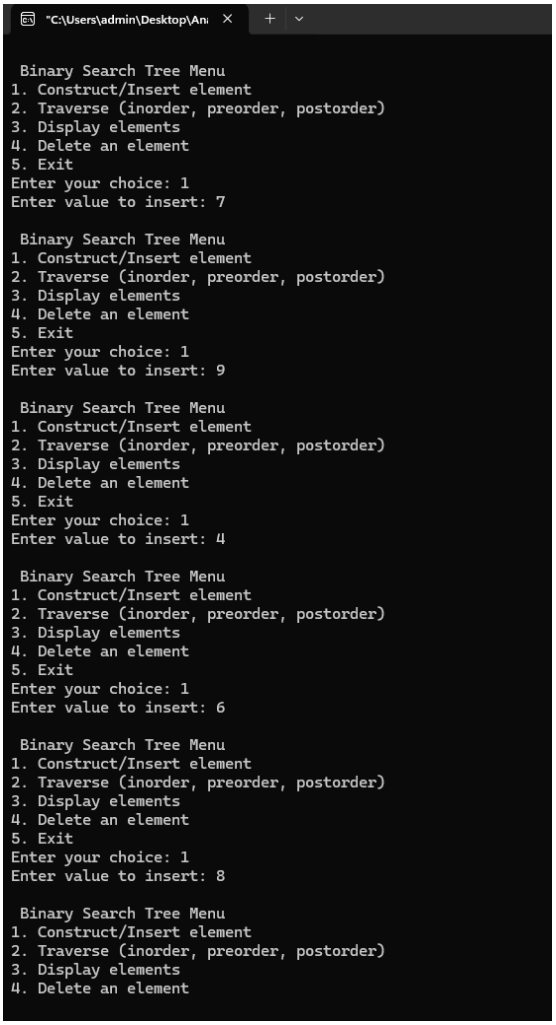
```
#include <stdio.h>
#include <stdlib.h>

struct node {
   int data;
   struct node *left;
   struct node *right;
};

struct node* create(int value) {
   struct node* newnode = (struct node*)malloc(sizeof(struct node));
   if (newnode == NULL) {
      printf("Memory not allocated\n");
      exit(1);
   }
   newnode->data = value;
   newnode->left = NULL;
   newnode->right = NULL;
   return newnode;
}

struct node* insert(struct node* root, int value) {
   if (root == NULL)
      return create(value);
   if (value < root->data)
      root->left = insert(root->left, value);
   else if (value > root->data)
      root->right = insert(root->right, value);
   return root;
}

struct node* find_min(struct node* ptr) {
   struct node* current = ptr;
   while (current && current->left != NULL)
      current = current->left;
   return current;
}

struct node* delete_node(struct node* root, int key) {
```

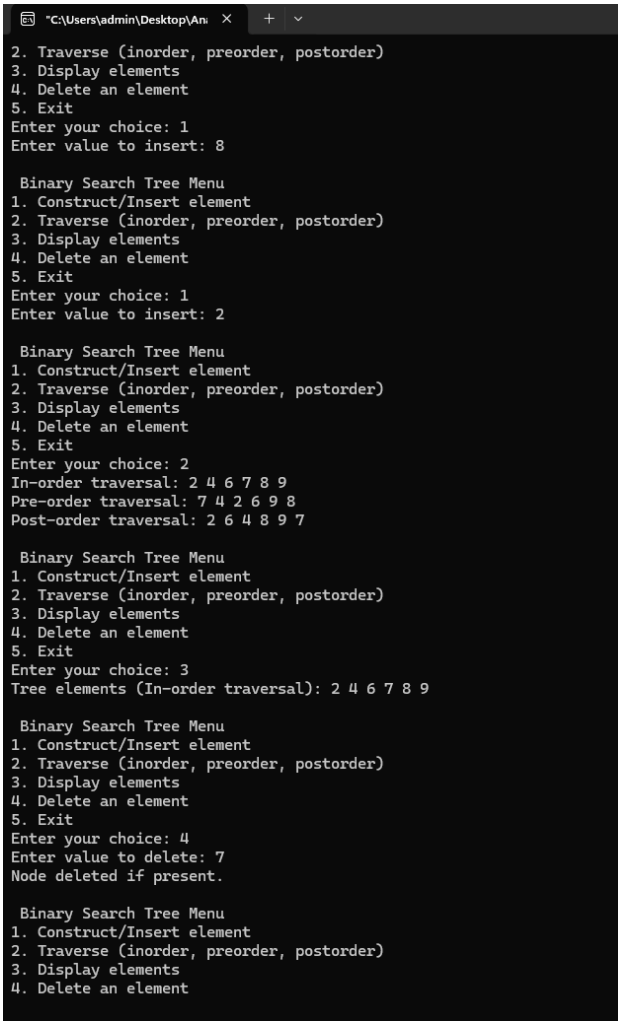
```
    if (root == NULL)
        return root;

    if (key < root->data)
        root->left = delete_node(root->left, key);
    else if (key > root->data)
        root->right = delete_node(root->right, key);
    else {
        if (root->left == NULL) {
            struct node* temp = root->right;
            free(root);
            return temp;
        }
        else if (root->right == NULL) {
            struct node* temp = root->left;
            free(root);
            return temp;
        }
        struct node* temp = find_min(root->right);
        root->data = temp->data;
        root->right = delete_node(root->right, temp->data);
    }
    return root;
}

void inorder_tra(struct node* root) {
    if (root != NULL) {
        inorder_tra(root->left);
        printf("%d ", root->data);
        inorder_tra(root->right);
    }
}

void preorder_tra(struct node* root) {
    if (root != NULL) {
        printf("%d ", root->data);
        preorder_tra(root->left);
        preorder_tra(root->right);
    }
}

void postorder_tra(struct node* root) {
    if (root != NULL) {
        postorder_tra(root->left);
        postorder_tra(root->right);
        printf("%d ", root->data);
    }
```

```
}

void display(struct node* root) {
   printf("Tree elements (In-order traversal): ");
   inorder_tra(root);
   printf("\n");
}

int main() {
   struct node* root = NULL;
   int choice, value;

   do {
      printf("\n Binary Search Tree Menu \n");
      printf("1. Construct/Insert element\n");
      printf("2. Traverse (inorder, preorder, postorder)\n");
      printf("3. Display elements\n");
      printf("4. Delete an element\n");
      printf("5. Exit\n");
      printf("Enter your choice: ");
      scanf("%d", &choice);

      switch (choice) {
         case 1:
            printf("Enter value to insert: ");
            scanf("%d", &value);
            root = insert(root, value);
            break;

         case 2:
            printf("In-order traversal: ");
            inorder_tra(root);
            printf("\n");
            printf("Pre-order traversal: ");
            preorder_tra(root);
            printf("\n");
            printf("Post-order traversal: ");
            postorder_tra(root);
            printf("\n");
            break;

         case 3:
            display(root);
            break;

         case 4:
            printf("Enter value to delete: ");
```

```
            scanf("%d", &value);
            root = delete_node(root, value);
            printf("Node deleted if present.\n");
            break;

        case 5:
            printf("Exiting program.\n");
            break;

        default:
            printf("Invalid choice. Please try again.\n");
      }
   } while (choice != 5);

   return 0;
}
```

Output:






"C:\Users\admin\Desktop\An
Pre-order traversal: 7 4 2 6 9 8
Post-order traversal: 2 6 4 8 9 7
Binary Search Tree Menu
1. Construct/Insert element
2. Traverse (inorder, preorder, postorder)
3. Display elements
4. Delete an element
5. Exit
Enter your choice: 3
Tree elements (In-order traversal): 2 4 6 7 8 9
Binary Search Tree Menu
1. Construct/Insert element
2. Traverse (inorder, preorder, postorder)
3. Display elements
4. Delete an element
5. Exit
Enter your choice: 4
Enter value to delete: 7
Node deleted if present.
Binary Search Tree Menu
1. Construct/Insert element
2. Traverse (inorder, preorder, postorder)
3. Display elements
4. Delete an element
5. Exit
Enter your choice: 3
Tree elements (In-order traversal): 2 4 6 8 9
Binary Search Tree Menu
1. Construct/Insert element
2. Traverse (inorder, preorder, postorder)
3. Display elements
4. Delete an element
5. Exit
Enter your choice: 5
Exiting program.
Process returned 0 (0x0) execution time : 66.793 s
Press any key to continue.

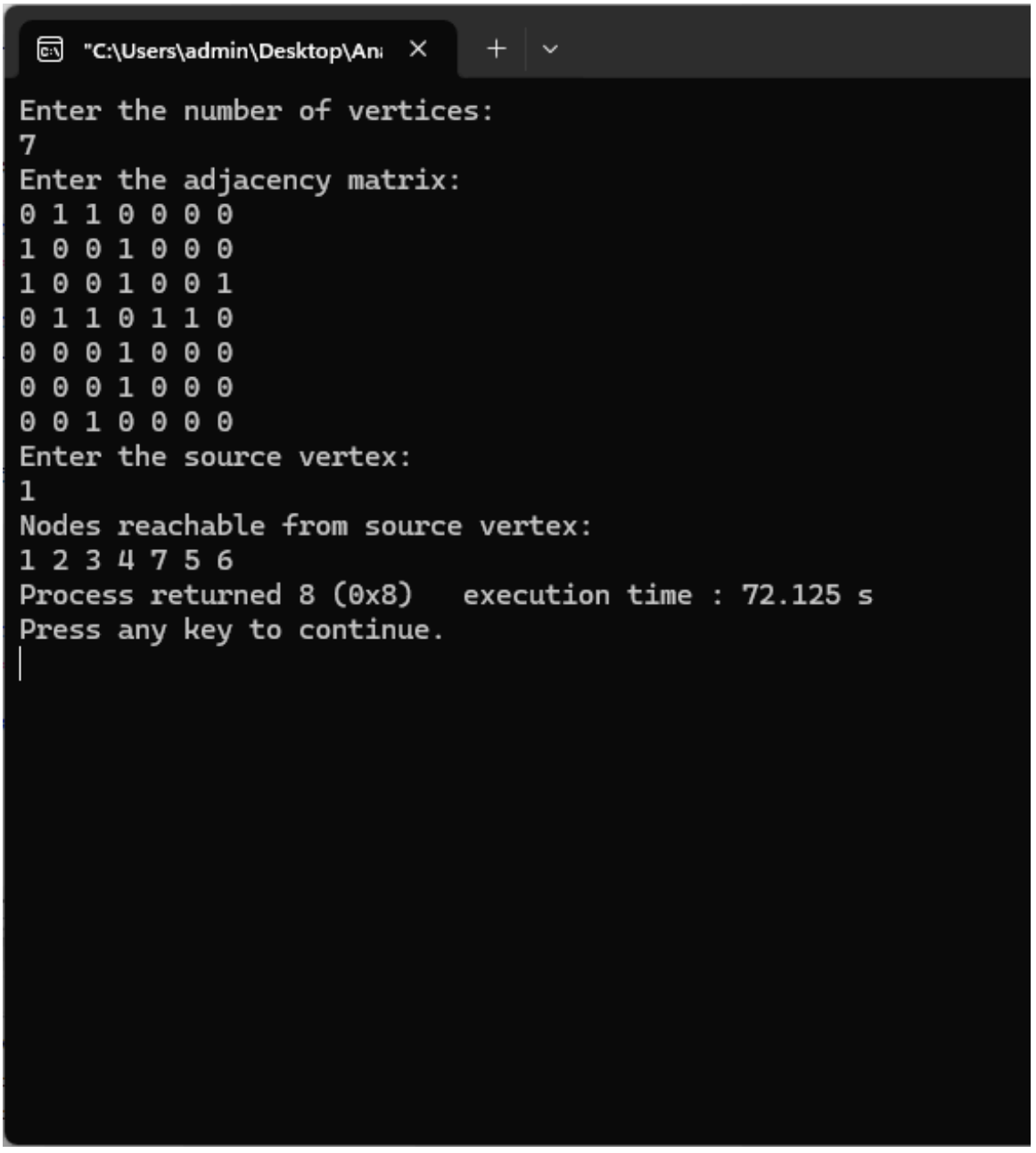
Lab program 9a:

Write a program to traverse a graph using BFS method.

```
#include <stdio.h>

void bfs(int);

int a[10][10], vis[10], n;

void main()
{
   int i, j, src;

   printf("Enter the number of vertices:\n");
   scanf("%d", &n);

   printf("Enter the adjacency matrix:\n");
   for (i = 1; i <= n; i++)
   {
      for (j = 1; j <= n; j++)
      {
         scanf("%d", &a[i][j]);
      }
      vis[i] = 0;
   }

   printf("Enter the source vertex:\n");
   scanf("%d", &src);

   printf("Nodes reachable from source vertex:\n");
   bfs(src);
}

void bfs(int v)
{
   int q[10], f = 1, r = 1, u, i;

   q[r] = v;
   vis[v] = 1;

   while (f <= r)
   {
      u = q[f];
      printf("%d ", u);

      for (i = 1; i <= n; i++)
```

```
        {
            if (a[u][i] == 1 && vis[i] == 0)
            {
                vis[i] = 1;
                r++;
                q[r] = i;
            }
        }
        f++;
    }
}
```

Output:



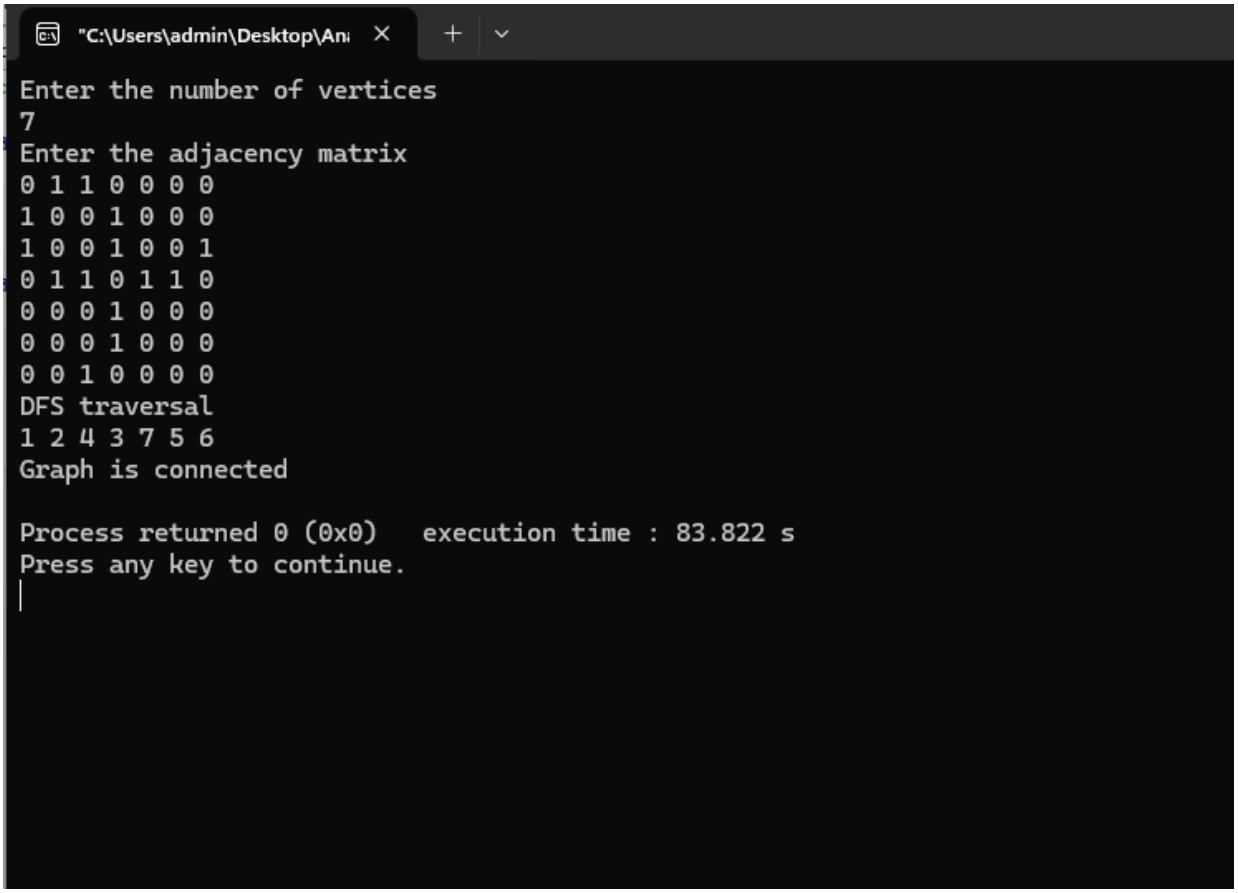
Lab program 9b:

Write a program to check whether given graph is connected or not using DFS method.

```
#include <stdio.h>

void dfs(int);

int n, i, j;
int a[10][10], vis[10], count;

void main()
{
   printf("Enter the number of vertices\n");
   scanf("%d", &n);

   printf("Enter the adjacency matrix\n");
   for (i = 1; i <= n; i++)
   {
      for (j = 1; j <= n; j++)
      {
         scanf("%d", &a[i][j]);
      }
      vis[i] = 0;
   }

   printf("DFS traversal\n");
   for (i = 1; i <= n; i++)
   {
      if (vis[i] == 0)
         dfs(i);
   }

   for (i = 1; i <= n; i++)
   {
      if (vis[i])
      {
         count++;
      }
   }

   if (count == n)
      printf("\nGraph is connected\n");
   else
      printf("\nGraph is not connected\n");
}
```

```
void dfs(int v)
{
   vis[v] = 1;
   printf("%d ", v);

   for (j = 1; j <= n; j++)
   {
      if (a[v][j] == 1 && vis[j] == 0)
      {
         dfs(j);
      }
   }
}
```

Output:



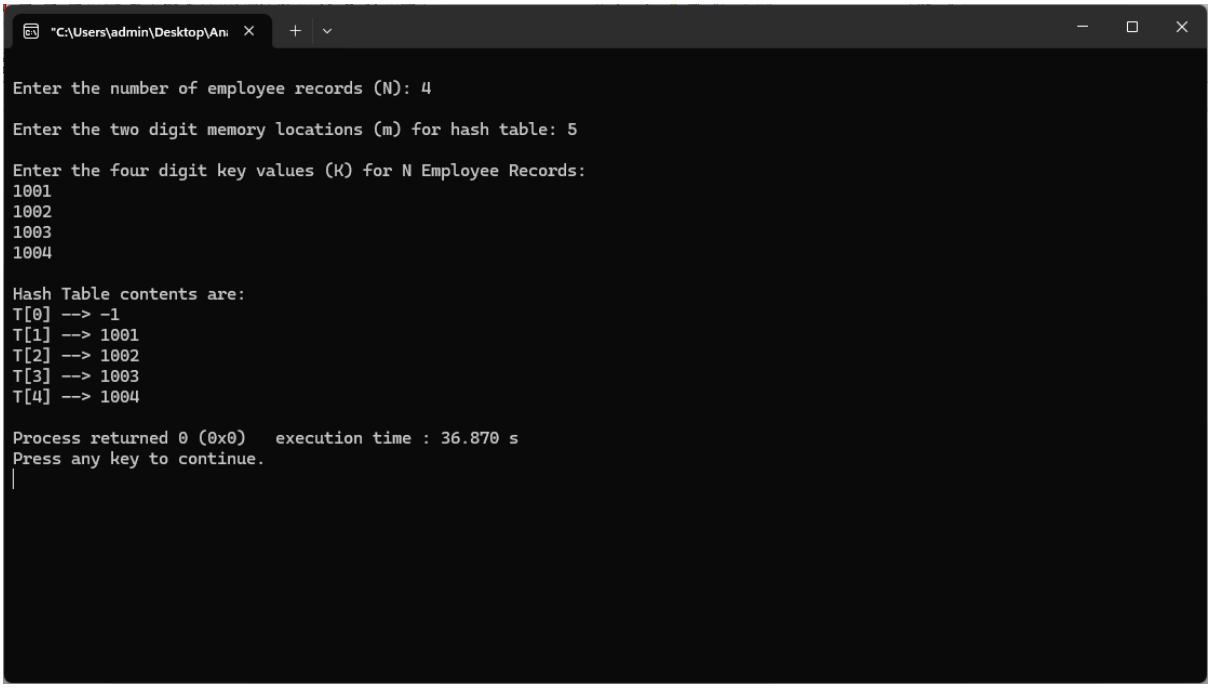
Lab program 10:

Given a File of N employee records with a set K of Keys(4- digit) which uniquely determine the records in file F. Assume that file F is maintained in memory by a Hash Table (HT) of m memory locations with L as the set of memory addresses (2-digit) of locations in HT. Let the keys in K and addresses in L are integers. Design and develop a Program in C that uses Hash function H: K -> L as H(K)=K mod m (remainder method), and implement hashing technique to map a given key K to the address space L. Resolve the collision (if any) using linear probing.

```
#include <stdio.h>
#include <stdlib.h>

int key[20], n, m;
int *ht, index;
int count = 0;

void insert(int key)
{
   index = key % m;
   while (ht[index] != -1)
   {
      index = (index + 1) % m;
   }
   ht[index] = key;
   count++;
}

void display()
{
   int i;
   if (count == 0)
   {
      printf("\nHash Table is empty");
      return;
   }

   printf("\nHash Table contents are:\n");
   for (i = 0; i < m; i++)
      printf("T[%d] --> %d\n", i, ht[i]);
}

int main()
```

```
{
   int i;

   printf("\nEnter the number of employee records (N): ");
   scanf("%d", &n);

   printf("\nEnter the two digit memory locations (m) for hash table: ");
   scanf("%d", &m);

   ht = (int *)malloc(m * sizeof(int));
   for (i = 0; i < m; i++)
      ht[i] = -1;

   printf("\nEnter the four digit key values (K) for N Employee Records:\n");
   for (i = 0; i < n; i++)
      scanf("%d", &key[i]);

   for (i = 0; i < n; i++)
   {
      if (count == m)
      {
         printf("\nHash table is full. Cannot insert the record %d key", i + 1);
         break;
      }
      insert(key[i]);
   }

   display();
   return 0;
}
```

Output:


"C:\Users\admin\Desktop\An
Enter the number of employee records (N): 4
Enter the two digit memory locations (m) for hash table: 5
Enter the four digit key values (K) for N Employee Records:
1001
1002
1003
1004
Hash Table contents are:
T[0] --> -1
T[1] --> 1001
T[2] --> 1002
T[3] --> 1003
T[4] --> 1004
Process returned 0 (0x0) execution time : 36.870 s
Press any key to continue.